

Tarea 2: Números Pseudoaleatorios y Métodos de Monte Carlo

Sofía Sandoval

Curso Libre de Configuración 2: Análisis de Datos
Facultad de Ingeniería, Universidad del Istmo de Guatemala

15 de septiembre de 2026

Resumen

Se implementaron desde cero cinco generadores de números pseudoaleatorios (LCG, cuadrados medios, Mersenne Twister, Blum Blum Shub y RANDU) y se sometieron a cuatro pruebas de hipótesis, a una prueba espectral visual y a una comparación contra las librerías `random` y `secrets`. La implementación propia de MT19937 reproduce bit a bit la salida de `random.random()`, y RANDU reproduce exactamente los 15 planos descritos por Marsaglia. Con esos generadores se construyó un estimador de Monte Carlo, se verificó la tasa de convergencia $\mathcal{O}(N^{-1/2})$ a lo largo de cinco órdenes de magnitud en N , desde 10^1 hasta 10^6 , se midió la maldición de la dimensionalidad estimando el volumen de la bola unitaria hasta $d = 20$, y se aplicaron dos técnicas de reducción de varianza que bajaron la varianza del estimador hasta 318.55 veces. El trabajo cierra reproduciendo la falla de RANDU dentro de un cálculo de Monte Carlo en tres dimensiones.

Índice

1. Introducción	3
2. Parte 1: Generadores de números pseudoaleatorios	4
2.1. Las cuatro pruebas de hipótesis	4
2.2. Generador de congruencia lineal (LCG)	5
2.2.1. Fundamento matemático	5
2.2.2. Pseudocódigo	6
2.2.3. Ficha comparativa	6
2.2.4. Resultados de la implementación	6
2.3. Método de cuadrados medios (von Neumann)	8
2.3.1. Fundamento matemático	8
2.3.2. Pseudocódigo	8
2.3.3. Ficha comparativa	9
2.3.4. Resultados de la implementación	9
2.4. Mersenne Twister (MT19937)	10
2.4.1. Fundamento matemático	10
2.4.2. Pseudocódigo	11
2.4.3. Ficha comparativa	12
2.4.4. Resultados de la implementación	12
2.5. Blum Blum Shub	15
2.5.1. Fundamento matemático	15
2.5.2. Pseudocódigo	15
2.5.3. Ficha comparativa	16
2.5.4. Resultados de la implementación	16
2.6. RANDU	17
2.6.1. Fundamento matemático	17
2.6.2. Pseudocódigo	18
2.6.3. Ficha comparativa	18
2.6.4. Resultados de la implementación	18
2.7. Comparación global de los cinco generadores	22
3. Parte 2: Métodos de Monte Carlo	23
3.1. Fundamento: el estimador y su error	23
3.1.1. La integral como valor esperado y el insesgamiento	23
3.1.2. Varianza y la tasa de convergencia	24
3.1.3. Intervalo de confianza del 95 %	24
3.2. Calentamiento: integrales en una dimensión	25
3.3. Alta dimensión y la maldición de la dimensionalidad	27
3.4. Reducción de varianza	28
3.4.1. Variables antitéticas	29
3.4.2. Variables de control	29
3.4.3. Resultados	30
3.5. Cuando el generador arruina el resultado: el caso RANDU	31
4. Conclusiones	34
Referencias	35

1. Introducción

Una computadora no sabe hacer azar de verdad, y ése es el punto de partida de toda la tarea. Lo que hace un generador de números pseudoaleatorios (PRNG) es iterar una función $X_{n+1} = T(X_n)$ sobre un estado interno finito y entregar alguna transformación de ese estado. Como el estado vive en un conjunto finito, la sucesión tiene que repetirse tarde o temprano: apenas un estado vuelve a aparecer, toda la historia que sigue se repite igualita. La diferencia con un generador verdaderamente aleatorio (TRNG), que mide ruido físico, es que el PRNG se puede repetir a voluntad. Suena a defecto, y en criptografía lo es, pero en simulación es justo lo que uno quiere: sin eso no habría manera de repetir un experimento.

Un buen PRNG tiene que cumplir dos cosas a la vez. La primera es que sus salidas se vean uniformes en $[0, 1)$. La segunda, que es la difícil y la que más se olvida, es que sean independientes: que conocer $u_1, \dots, u_n$ no ayude en nada a adivinar u_{n+1} . Lo que se va a ver aquí es que la primera es fácil de cumplir y fácil de verificar, mientras que la segunda puede fallar de maneras que ninguna prueba unidimensional alcanza a ver. RANDU es el ejemplo clásico y aparece dos veces en este documento: primero como el generador que pasa las cuatro pruebas de hipótesis sin despeinarse, y después como el generador que arruina un cálculo de Monte Carlo.

La segunda parte usa estos mismos generadores para alimentar métodos de Monte Carlo, que estiman cantidades deterministas escribiéndolas como un valor esperado y promediando muestras. La Ley de los Grandes Números garantiza que el promedio llega a donde tiene que llegar, y el Teorema del Límite Central dice qué tan rápido. La moraleja, que se arma al final con números, es que un método de Monte Carlo nunca va a ser mejor que la fuente de aleatoriedad que lo alimenta.

Organización y reproducibilidad

Cada generador se presenta completo y por separado, siempre en el mismo orden: fundamento matemático, pseudocódigo, ficha comparativa de velocidad, período, seguridad y casos de uso, y finalmente los resultados de su implementación (histograma, pruebas de hipótesis y, cuando corresponde, prueba espectral y comparación con la librería estándar).

Todos los generadores reciben su semilla como argumento del constructor. Las semillas usadas son **141903** para LCG, cuadrados medios, Mersenne Twister y RANDU, y **14773** para Blum Blum Shub, que necesita una semilla coprima con su módulo. RANDU exige semilla impar y 141903 lo es. Todas las figuras y tablas de la Parte 1 salen de una sola ejecución de `src/part1_generators.py` con $N = 100,000$ valores por generador.

2. Parte 1: Generadores de números pseudoaleatorios

Los cinco algoritmos están implementados en `src/prngs.py` sin usar `random` ni `numpy.random` como motor. Todos exponen la misma interfaz: `next_int()` devuelve el entero crudo del estado y `random()` lo normaliza a $[0, 1)$ dividiendo entre el módulo. Las cuatro pruebas de hipótesis están en `src/stats_tests.py`, también desde cero: como `scipy` no estaba disponible, se programaron la función gamma incompleta y la distribución de Kolmogorov siguiendo *Numerical Recipes*, y se verificaron contra valores de tabla.

2.1. Las cuatro pruebas de hipótesis

Antes de entrar a cada generador conviene fijar qué se les va a aplicar. Dos pruebas miden uniformidad y dos miden independencia serial. En las cuatro, H_0 es “la muestra se comporta como *i.i.d.* Unif(0, 1)” y se rechaza al 5 % cuando $p < 0,05$.

Chi-cuadrado. Se parte $[0, 1)$ en $k = 10$ celdas de igual ancho. Bajo H_0 cada celda espera $E = N/k$ observaciones y

$$X^2 = \sum_{j=1}^k \frac{(O_j - E)^2}{E} \xrightarrow{d} \chi_{k-1}^2, \quad (1)$$

con $k - 1$ grados de libertad porque no se estima ningún parámetro a partir de los datos: la hipótesis nula ya fija las probabilidades de celda en $1/k$.

Kolmogorov–Smirnov. Compara la función de distribución empírica contra la teórica midiendo su máxima separación vertical, $D = \sup_x |F_N(x) - x|$. Sobre la muestra ordenada $u_{(1)} \leq \dots \leq u_{(N)}$ esto se calcula como

$$D = \max \left(\max_i \left(\frac{i}{N} - u_{(i)} \right), \max_i \left(u_{(i)} - \frac{i-1}{N} \right) \right), \quad (2)$$

y el p -valor usa la aproximación asintótica $Q((\sqrt{N} + 0,12 + 0,11/\sqrt{N}) D)$ con $Q(\lambda) = 2 \sum_{k \geq 1} (-1)^{k-1} e^{-2k^2 \lambda^2}$. A diferencia de χ^2 , KS no necesita agrupar en celdas, así que detecta desviaciones locales que una celda ancha podría promediar.

Autocorrelación de retardo 1. Es la primera prueba de independencia: contrasta $H_0 : \rho_1 = 0$ con el coeficiente muestral r_1 entre u_i y u_{i+1} . Bajo H_0 y para N grande, $\sqrt{N} r_1 \xrightarrow{d} \mathcal{N}(0, 1)$.

Prueba de rachas. Codifica cada valor como “arriba” o “abajo” de la mediana y cuenta cuántos bloques consecutivos del mismo signo hay. Con n_1 valores arriba y n_2 abajo,

$$\mathbb{E}[R] = \frac{2n_1 n_2}{n_1 + n_2} + 1, \quad \text{Var}(R) = \frac{2n_1 n_2 (2n_1 n_2 - n_1 - n_2)}{(n_1 + n_2)^2 (n_1 + n_2 - 1)}, \quad (3)$$

y $z = (R - \mathbb{E}[R]) / \sqrt{\text{Var}(R)}$ se compara contra una normal estándar. Detecta algo que la autocorrelación no ve: agrupamientos o alternancias que no son lineales.

Vale la pena fijarse desde ya en algo: las cuatro son pruebas de una o dos dimensiones. Ninguna mira ternas. Esa limitación, que parece un detalle, es exactamente la razón por la que hace falta la prueba espectral de la sección 2.6.

2.2. Generador de congruencia lineal (LCG)

2.2.1. Fundamento matemático

Un LCG genera la sucesión de enteros

$$X_{n+1} = (aX_n + c) \text{ mód } m, \quad U_n = \frac{X_n}{m}, \quad (4)$$

con módulo m , multiplicador a , incremento c y semilla X_0 , todos enteros con $0 \leq X_0, a, c < m$. Cuando $c \neq 0$ se le llama *mixto*; cuando $c = 0$, *multiplicativo* (ese es el caso de RANDU, sección 2.6).

Forma cerrada. La recurrencia lineal se puede resolver explícitamente. Aplicándola dos veces, $X_2 = a(aX_0 + c) + c = a^2X_0 + c(a + 1)$, y por inducción

$$X_n = \left(a^n X_0 + c \frac{a^n - 1}{a - 1} \right) \text{ mód } m \quad (a \neq 1), \quad (5)$$

donde el cociente se interpreta módulo m . La consecuencia práctica es que un LCG no tiene memoria: cualquier salida futura queda determinada por X_0 y por tres constantes.

Periodicidad. El estado vive en $\{0, 1, \dots, m-1\}$, que es finito, así que por el principio del palomar entre $X_0, X_1, \dots, X_m$ hay dos iguales. En cuanto $X_i = X_j$ con $i < j$, la sucesión se repite con período $j - i$, y por lo tanto el período nunca puede exceder m . Además, si $\text{mcd}(a, m) = 1$ el mapa $X \mapsto aX + c$ es una biyección módulo m , de modo que la sucesión es *puramente* periódica: no tiene una cola inicial que no se vuelva a visitar.

Teorema de Hull–Dobell. El período es exactamente m para toda semilla si y sólo si se cumplen estas tres condiciones:

- (I) $\text{mcd}(c, m) = 1$;
- (II) $a - 1$ es divisible por todos los factores primos de m ;
- (III) si $4 \mid m$, entonces $4 \mid (a - 1)$.

La idea detrás de las condiciones es que la órbita recorra todo $\mathbb{Z}_m$: la primera obliga a que el incremento genere el grupo aditivo, y las otras dos obligan a que $a - 1$ sea lo bastante divisible como para que la parte multiplicativa no encierre la órbita en un subgrupo propio.

Los parámetros usados aquí son los de *Numerical Recipes*: $a = 1664525$, $c = 1013904223$ y $m = 2^{32}$. Se verifican las tres condiciones: c es impar, así que $\text{mcd}(c, 2^{32}) = 1$; el único factor primo de m es 2 y $a - 1 = 1664524$ es par; y $1664524 = 4 \cdot 416131$ es divisible entre 4. El período es entonces exactamente $2^{32} \approx 4,29 \times 10^9$.

Estructura reticular. El defecto estructural de todo LCG es que sus d -tuplas consecutivas no llenan el cubo $[0, 1)^d$, sino que caen en una retícula. El teorema de Marsaglia acota el número de hiperplanos paralelos que las contienen por

$$N_d \leq (d! m)^{1/d}. \quad (6)$$

Para $d = 3$ y $m = 2^{32}$ esto da a lo sumo $(6 \cdot 2^{32})^{1/3} \approx 2954$ planos. La gracia está en que un LCG bien parametrizado se acerca a esa cota, y con miles de planos los huecos quedan tan chiquitos que en una figura uno no los distingue; uno mal parametrizado se queda muy por debajo y ahí sí se ve el desastre. Ésa es toda la diferencia entre este generador y RANDU, que se queda en apenas 15 planos.

Los bits bajos son un problema aparte. Con $m = 2^{32}$ hay otro defecto que conviene conocer: el bit j -ésimo menos significativo de X_n tiene período 2^{j+1} . O sea, el último bit alterna con período 2, el siguiente con período 4, y así. Entonces, si uno quiere un entero chico, no debe hacer $X \% k$, porque estaría usando justo los bits peores; hay que tomar los bits altos, multiplicando y truncando.

2.2.2. Pseudocódigo

Algoritmo 2.1 | Generador de congruencia lineal

Entrada: semilla X_0 ; parámetros a, c, m ; cantidad N .

Salida: sucesión $u_1, \dots, u_N$ de valores en $[0, 1)$.

```

1 Verificar que  $0 \leq a, c < m$  y que  $m > 0$ .
2  $X \leftarrow X_0 \bmod m$  (reduce la semilla al espacio de estados)
3 para  $i = 1, 2, \dots, N$ :
    3.1  $X \leftarrow (a \cdot X + c) \bmod m$ 
    3.2  $u_i \leftarrow X/m$ 
4 devolver  $u_1, \dots, u_N$ 
```

2.2.3. Ficha comparativa

Tabla 1: Características del LCG.

Velocidad	Muy alta. Cada valor cuesta una multiplicación, una suma y un módulo, y con $m = 2^{32}$ el módulo es un truncamiento de bits. Medido: 3,045,229 valores por segundo en Python puro.
Período	2^{32} para toda semilla, garantizado por Hull–Dobell. Los bits bajos tienen períodos mucho más cortos.
Seguridad	Nula. Con dos o tres salidas consecutivas se resuelve el sistema para a, c y m , y a partir de ahí toda la sucesión es predecible.
Casos de uso	Simulación no crítica, videojuegos, <code>rand()</code> de C, sistemas embebidos donde el costo por número importa más que la calidad.

2.2.4. Resultados de la implementación

Tabla 2: Pruebas de hipótesis para LCG sobre $N = 100,000$ valores. Se rechaza H_0 al 5 % si $p < 0,05$.

Prueba	Mide	Estadístico	p -valor	Decision
χ^2 ($k = 10$)	uniformidad	6.17	0.7223	no rechaza H_0
Kolmogorov–Smirnov	uniformidad	0.00312	0.2830	no rechaza H_0
Autocorrelacion lag-1	independencia	+0.00119	0.7075	no rechaza H_0
Rachas	independencia	49849	0.3364	no rechaza H_0

En el histograma no hay ninguna celda que se despegue de la línea punteada, y las cuatro pruebas dicen lo mismo. La chi-cuadrado da $X^2 = 6,17$ con $p = 0,7223$, lejísimos del umbral de 0,05, así que no hay nada que sugiera que las diez celdas se están apartando de lo esperado. Kolmogorov–Smirnov da

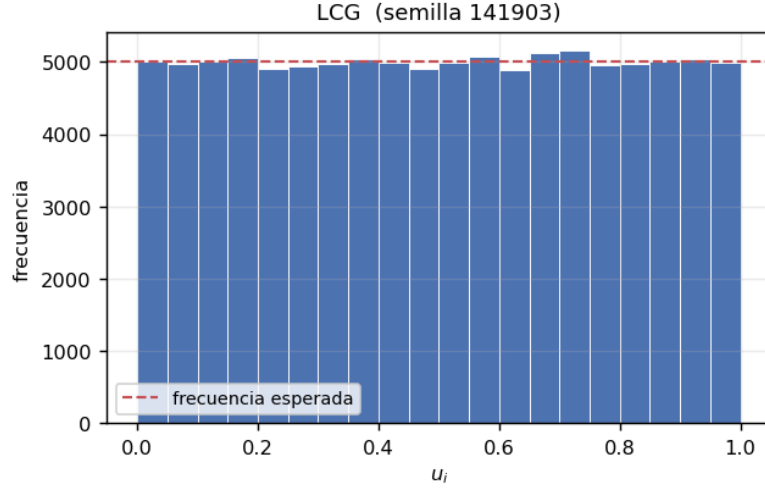


Figura 1: Histograma de $N = 100,000$ valores del LCG con semilla 141903, en 20 celdas. La línea punteada marca la frecuencia esperada bajo uniformidad.

$D = 0,00312$ con $p = 0,2830$: la separación máxima entre la acumulada empírica y la teórica anda en el orden de 10^{-3} , que es lo normal con $N = 100,000$. Del lado de la independencia, la autocorrelación de retardo 1 sale $r_1 = +0,00119$ con $p = 0,7075$, o sea indistinguible de cero, y la prueba de rachas cuenta $R = 49,849$ rachas con $p = 0,3364$, prácticamente el valor esperado. Ninguna de las cuatro rechaza H_0 , y este generador se porta bien.

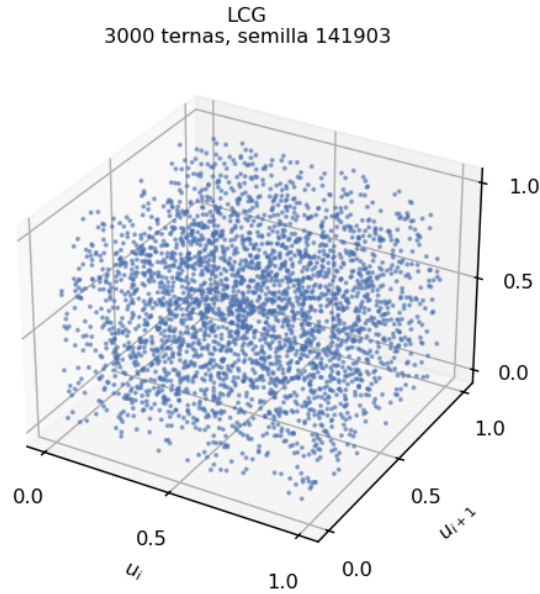


Figura 2: Ternas consecutivas (u_i, u_{i+1}, u_{i+2}) del LCG en el cubo $[0, 1]^3$ (3,000 ternas).

En la figura 2 la nube llena el cubo y no se le ve ninguna estructura. Ojo con la interpretación: eso no quiere decir que la retícula no exista. Por la cota (6) sabemos que los planos están ahí, pero al ser del orden de miles quedan separados por distancias de 10^{-3} , mucho más finas de lo que se puede distinguir en una figura con 3,000 puntos. Comparado con RANDU (sección 2.6), la diferencia es puramente

de cantidad: es el mismo tipo de defecto, sólo que tres órdenes de magnitud más disimulado.

2.3. Método de cuadrados medios (von Neumann)

2.3.1. Fundamento matemático

Propuesto por von Neumann en 1949, es el primer PRNG de la historia. Se toma un estado de k dígitos decimales, se eleva al cuadrado para obtener hasta $2k$ dígitos, se rellena con ceros a la izquierda hasta tener exactamente $2k$, y se conservan los k dígitos centrales:

$$X_{n+1} = \left\lfloor \frac{X_n^2}{10^{k/2}} \right\rfloor \bmod 10^k, \quad U_n = \frac{X_n}{10^k}. \quad (7)$$

La idea intuitiva es que los dígitos del centro de un cuadrado dependen de todos los dígitos del original, así que “mezclan” bien.

Por qué aquí no hay ninguna garantía de período. La diferencia de fondo con el LCG es que el mapa (7) **no es una biyección**. Varios estados distintos pueden terminar con los mismos dígitos centrales, y en cuanto eso pasa la órbita se va encogiendo. Un mapa cualquiera sobre un conjunto de n elementos arma lo que se llama un grafo funcional en forma de ρ : una cola de estados que se visitan una sola vez y después un ciclo del que ya no se sale nunca. Para un mapa tomado al azar, tanto la cola como el ciclo miden en promedio del orden de $\sqrt{\pi n/8}$, o sea $\mathcal{O}(10^{k/2})$ en vez del 10^k que uno esperaría viendo el tamaño del espacio de estados. Se pierde la mitad de los dígitos de período nada más por la forma que tiene el mapa.

Y encima el cero se traga todo. Hay un modo de falla todavía peor. Si en algún paso los dígitos centrales salen todos cero, entonces $X_n = 0$, y como $0^2 = 0$ el generador se queda en cero para siempre. Es un punto fijo sin salida: el generador no se vuelve periódico, se muere. Y hay trampas parecidas, como los ciclos cortos de longitud 100 que aparecieron probando otras semillas.

Lo que pasó con nuestra semilla. Con $k = 8$ dígitos y semilla 141903, la sucesión cae en el punto fijo $X = 0$ en el paso $i = 8,864$, y el ciclo que queda tiene longitud 1. Como la muestra que pide la tarea es de $N = 100,000$ valores, resulta que el 91.1% de la muestra son ceros idénticos. Repitiendo el experimento con 200 semillas consecutivas a partir de 141903, el tramo útil antes de repetir un estado va de 22 a 16,072 pasos, con media 4,625. Esa dispersión gigante es justamente el punto: el método no falla de manera predecible, falla distinto según la semilla que le toque, que para un generador es lo peor de los dos mundos.

2.3.2. Pseudocódigo

Algoritmo 2.2 | Método de cuadrados medios

Entrada: semilla X_0 de k dígitos, con k par; cantidad N .

Salida: sucesión $u_1, \dots, u_N$ de valores en $[0, 1)$.

-
- 1 $X \leftarrow X_0 \bmod 10^k$
 - 2 **para** $i = 1, 2, \dots, N$:
 - 2.1 $s \leftarrow$ representación decimal de X^2 , rellena con ceros a la izquierda hasta $2k$ dígitos
 - 2.2 $X \leftarrow$ el número formado por los k dígitos centrales de s

```

2.3  $u_i \leftarrow X/10^k$ 
2.4 si  $X = 0$ : el generador degeneró y todo lo que sigue será cero
3 devolver  $u_1, \dots, u_N$ 

```

2.3.3. Ficha comparativa

Tabla 3: Características del método de cuadrados medios.

Velocidad	Alta en teoría, pero opera sobre la representación decimal, que obliga a convertir a texto. Medido: 1,655,135 valores por segundo.
Período	Sin garantía alguna. Depende de la semilla y puede colapsar a cero. Con semilla 141903 el tramo útil fue de 8,864 pasos.
Seguridad	Nula.
Casos de uso	Sólo interés histórico y didáctico. Sirve justamente para mostrar por qué hace falta una teoría de períodos antes de confiar en un generador.

2.3.4. Resultados de la implementación

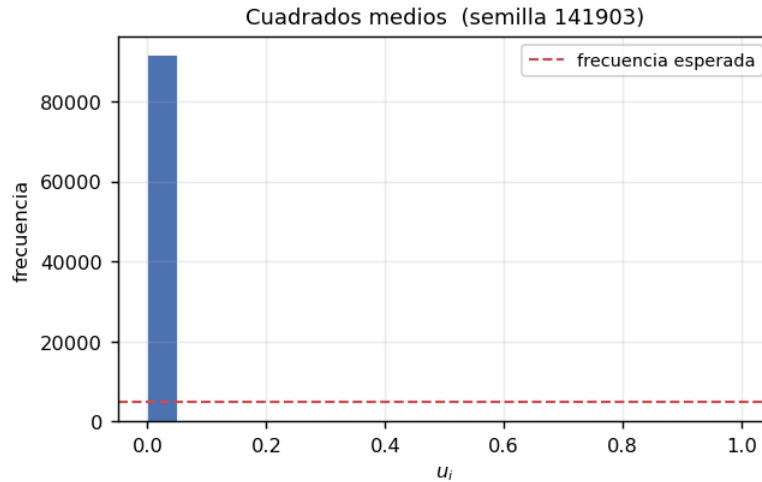


Figura 3: Histograma de $N = 100,000$ valores del método de cuadrados medios con semilla 141903. La barra de la primera celda se sale de la escala: el 91.1 % de la muestra son ceros.

Este histograma ni siquiera necesita estadística para leerse: casi toda la masa está parada en el cero. Las cuatro pruebas rechazan H_0 de forma catastrófica. La chi-cuadrado da $X^2 = 748369,88$ contra un valor crítico de 16,9, cuatro órdenes de magnitud arriba. KS da $D = 0,91138$, o sea que la acumulada empírica se despegue de la teórica casi tanto como es matemáticamente posible, porque D nunca puede pasar de 1. La autocorrelación sale $r_1 = +0,73241$, que era de esperarse: si la sucesión es una constante, cada valor predice perfecto al siguiente. Y la prueba de rachas cuenta $R = 0$ rachas, sencillamente porque nunca se cruza la mediana.

Aquí hay que separar dos cosas que las pruebas están mezclando. Aplicadas a los 100,000 valores, lo que miden no es la calidad de los números sino el colapso. Si en cambio se aplican sólo al tramo útil, o sea a los primeros $n = 8,864$ valores antes de que el generador se muera, el panorama cambia por

Tabla 4: Pruebas de hipotesis para Cuadrados medios sobre $N = 100,000$ valores. Se rechaza H_0 al 5 % si $p < 0,05$.

Prueba	Mide	Estadistico	p -valor	Decision
χ^2 ($k = 10$)	uniformidad	748369.88	$< 10^{-12}$	rechaza H_0
Kolmogorov–Smirnov	uniformidad	0.91138	$< 10^{-12}$	rechaza H_0
Autocorrelacion lag-1	independencia	+0.73241	$< 10^{-12}$	rechaza H_0
Rachas	independencia	0	$< 10^{-12}$	rechaza H_0

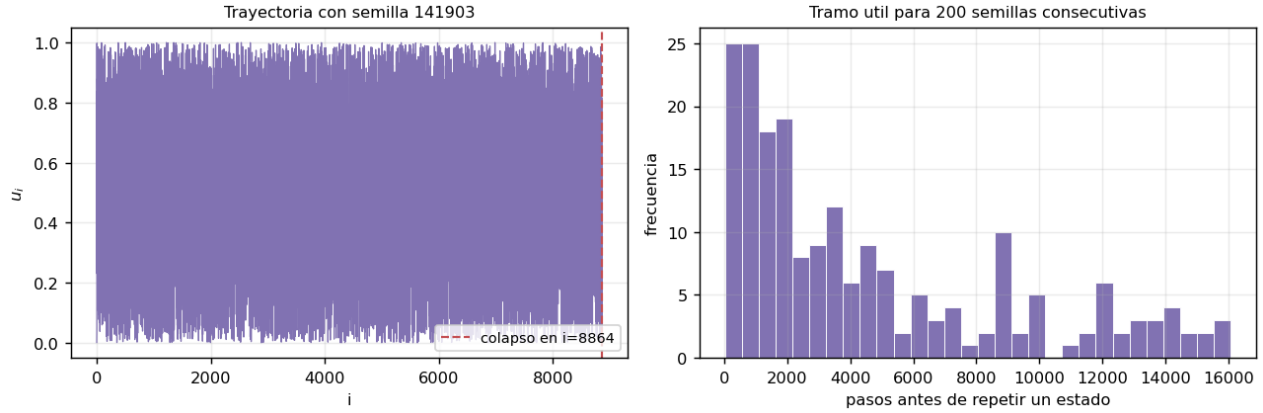


Figura 4: Izquierda: trayectoria del generador con semilla 141903; el colapso al punto fijo cero ocurre en $i = 8,864$. Derecha: distribución del largo del tramo útil para 200 semillas consecutivas.

completo: $X^2 = 5,31$ con $p = 0,257$ y $D = 0,0092$ con $p = 0,443$, y ya no se rechaza H_0 . Entonces la conclusión justa sobre el método de von Neumann no es que produzca números mal distribuidos, sino que produce pocos. Mientras dura se ve bastante decente; el problema es que se acaba.

2.4. Mersenne Twister (MT19937)

2.4.1. Fundamento matemático

MT19937 (Matsumoto y Nishimura, 1998) es el generador estándar de Python, R, MATLAB y NumPy. Es un *generador de Fibonacci retardado torcido*, y toda su teoría vive en el cuerpo $\mathbb{F}_2 = \{0, 1\}$: cada palabra de 32 bits se ve como un vector de $\mathbb{F}_2^{32}$ y todas las operaciones (XOR, desplazamientos, máscaras) son lineales sobre ese cuerpo.

La recurrencia. El estado son $n = 624$ palabras de $w = 32$ bits. La recurrencia combina dos palabras separadas por $m = 397$ posiciones:

$$\mathbf{x}_{k+n} = \mathbf{x}_{k+m} \oplus \left((\mathbf{x}_k^u \mid \mathbf{x}_{k+1}^l) A \right), \quad (8)$$

donde $\mathbf{x}_k^u$ es el bit más significativo de $\mathbf{x}_k$, $\mathbf{x}_{k+1}^l$ son los 31 bits menos significativos de $\mathbf{x}_{k+1}$, la barra vertical los concatena en una palabra de 32 bits, y A es la matriz compañera de 32×32 cuya acción se calcula sin multiplicar matrices:

$$\mathbf{y}A = \begin{cases} \mathbf{y} \gg 1 & \text{si el bit menos significativo de } \mathbf{y} \text{ es } 0, \\ (\mathbf{y} \gg 1) \oplus 0\mathbf{x}9908\mathbf{B0DF} & \text{en otro caso.} \end{cases} \quad (9)$$

El nombre “torcido” viene precisamente de esa matriz A , que es la torsión que rompe la simetría del retardo de Fibonacci.

De dónde sale el período. Toda la iteración es una aplicación lineal $\mathbf{s} \mapsto B\mathbf{s}$ sobre el espacio de estados $\mathbb{F}_2^{nw-r}$, donde $r = 31$ son los bits que se descartan de la primera palabra. La dimensión efectiva es

$$nw - r = 624 \times 32 - 31 = 19937. \quad (10)$$

El período del generador es el orden de B en el grupo lineal, y ese orden es $2^{19937} - 1$ si y sólo si el polinomio característico de B es *primitivo* sobre $\mathbb{F}_2$. Los autores eligieron los parámetros (n, m, r, a) justamente para que lo fuera, verificándolo con la factorización conocida de $2^{19937} - 1$, que es un primo de Mersenne: de ahí el nombre del generador. Que el número sea primo ayuda, porque entonces cualquier estado no nulo tiene automáticamente período completo.

Templado. La salida cruda de la recurrencia no se entrega directamente. Pasa antes por un *templado* (*tempering*), que es otra transformación lineal invertible sobre $\mathbb{F}_2^{32}$:

$$y \leftarrow y \oplus (y \gg 11), \quad y \leftarrow y \oplus ((y \ll 7) \& 0x9D2C5680), \quad (11)$$

$$y \leftarrow y \oplus ((y \ll 15) \& 0xEFC60000), \quad y \leftarrow y \oplus (y \gg 18). \quad (12)$$

El templado no cambia el período, porque es invertible; lo que hace es mejorar la equidistribución de los bits individuales.

Equidistribución. Ésta es la propiedad que de verdad importa y que separa a MT19937 de todo lo anterior. El generador es *equidistribuido a 32 bits de precisión en 623 dimensiones*: si se toman las 623-tuplas consecutivas de salidas y se miran sus 32 bits, cada patrón posible aparece el mismo número de veces en un período completo (salvo el caso del cero). Es decir, la retícula que en el LCG se veía en dimensión 3 aquí no aparece hasta después de la dimensión 623. Por eso pasa la prueba espectral en las dimensiones donde RANDU se hunde.

Lo que no es. Con todo lo bueno que tiene, MT19937 no sirve para criptografía. Como toda la recurrencia es lineal, a alguien le basta con ver 624 salidas seguidas para invertir el templado, reconstruir el estado completo y predecir todo lo que viene después. Es rapidísimo y estadísticamente impecable, pero completamente predecible si uno lo mira con las herramientas correctas.

2.4.2. Pseudocódigo

Algoritmo 2.3 | Mersenne Twister MT19937

Entrada: semilla s ; cantidad N . Constantes $n = 624$, $m = 397$, $A = 0x9908B0DF$.

Salida: sucesión $u_1, \dots, u_N$ de valores en $[0, 1)$.

1 Inicialización del estado:

1.1 $mt[0] \leftarrow s \bmod 2^{32}$

1.2 **para** $i = 1, \dots, n - 1$: $mt[i] \leftarrow (1812433253 \cdot (mt[i - 1] \oplus (mt[i - 1] \gg 30)) + i) \bmod 2^{32}$

1.3 $idx \leftarrow n$ (fuerza una torsión en la primera llamada)

2 **para** $i = 1, \dots, N$:

2.1 **si** $idx \geq n$, aplicar la torsión a todo el estado:

```

2.1.1 para  $j = 0, \dots, n - 1$ :
2.1.2    $y \leftarrow (mt[j] \& 0x80000000) | (mt[(j + 1) \bmod n] \& 0x7FFFFFFF)$ 
2.1.3    $mt[j] \leftarrow mt[(j + m) \bmod n] \oplus (y \gg 1)$ 
2.1.4   si  $y$  es impar:  $mt[j] \leftarrow mt[j] \oplus A$ 
2.1.5    $idx \leftarrow 0$ 
2.2    $y \leftarrow mt[idx]; \quad idx \leftarrow idx + 1$ 
2.3   Templado:  $y \leftarrow y \oplus (y \gg 11); \quad y \leftarrow y \oplus ((y \ll 7) \& 0x9D2C5680); \quad y \leftarrow y \oplus ((y \ll 15) \& 0xEFC60000); \quad y \leftarrow y \oplus (y \gg 18)$ 
2.4    $u_i \leftarrow y/2^{32}$ 
3   devolver  $u_1, \dots, u_N$ 

```

2.4.3. Ficha comparativa

Tabla 5: Características del Mersenne Twister.

Velocidad	Media. El estado de 624 palabras se recalcula por bloques, así que una de cada 624 llamadas es cara y las demás son baratas. Medido: 982,384 valores por segundo en Python puro; la versión en C de random es más de 20 veces más rápida.
Período	$2^{19937} - 1$, garantizado por la primitividad del polinomio característico. Para efectos prácticos, inagotable.
Seguridad	Nula. Con 624 salidas consecutivas se invierte el templado y se reconstruye el estado completo.
Casos de uso	Estándar de facto en simulación científica y estadística: Python, R, MATLAB, NumPy. Es la opción correcta por defecto cuando no se necesita seguridad criptográfica.

2.4.4. Resultados de la implementación

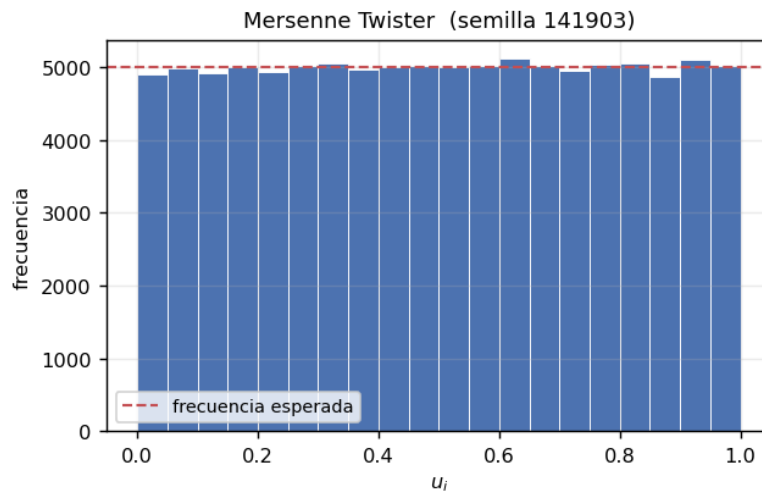


Figura 5: Histograma de $N = 100,000$ valores de la implementación propia de MT19937 con semilla 141903.

Tabla 6: Pruebas de hipotesis para Mersenne Twister sobre $N = 100,000$ valores. Se rechaza H_0 al 5 % si $p < 0,05$.

Prueba	Mide	Estadistico	p -valor	Decision
χ^2 ($k = 10$)	uniformidad	6.51	0.6879	no rechaza H_0
Kolmogorov–Smirnov	uniformidad	0.00303	0.3175	no rechaza H_0
Autocorrelacion lag-1	independencia	+0.00245	0.4393	no rechaza H_0
Rachas	independencia	49904	0.5396	no rechaza H_0

Las cuatro pruebas salen como uno esperaría del generador que todo el mundo usa de referencia. La chi-cuadrado da $X^2 = 6,51$ con $p = 0,6879$ y KS da $D = 0,00303$ con $p = 0,3175$, así que la muestra es indistinguible de una uniforme de verdad. En independencia, $r_1 = +0,00245$ con $p = 0,4393$ y $R = 49,904$ rachas con $p = 0,5396$. Ninguna rechaza H_0 . Y un detalle que vale la pena notar: los p -valores tampoco son sospechosamente altos, porque eso también sería mala señal; un p casi igual a 1 significaría que los datos se ajustan *demasiado* bien a la uniforme, que es lo que pasa cuando alguien los acomodó.

Validación de la implementación. Una implementación propia de MT19937 se puede verificar de manera mucho más fuerte que con pruebas estadísticas, porque el algoritmo es determinista y está publicado:

1. Con la semilla oficial 5489, las primeras salidas de 32 bits son 3499211612, 581869302, 3890346734, 3586334585, 545404204, que coinciden con el vector de referencia de los autores.
2. Implementando además el sembrado `init_by_array` que usa CPython y el empaquetado de 53 bits de `genrand_res53`, la implementación propia reproduce **bit a bit** la salida de `random.random()` para la misma semilla. La verificación automática sobre 100,000 valores da: idéntico = si.

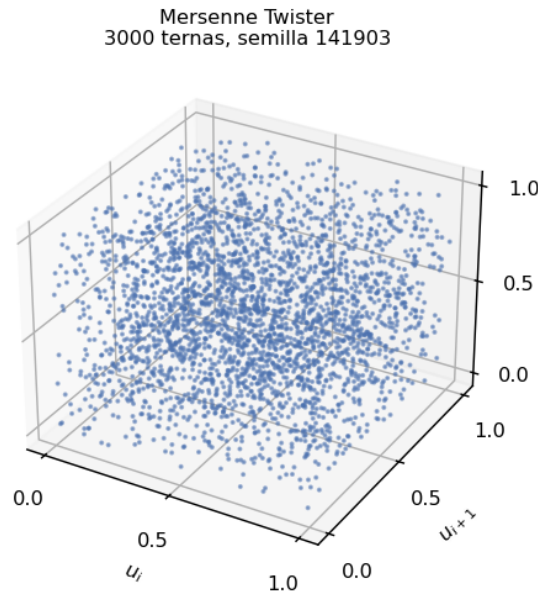


Figura 6: Ternas consecutivas del Mersenne Twister en el cubo $[0, 1]^3$.

La nube de ternas de la figura 6 no muestra estructura, y en este caso sabemos que no la hay hasta

dimensión 623, no sólo que no se ve.

Comparación con `random` y `secrets`. El generador elegido para el inciso (d) es este mismo. Las tres fuentes comparadas son:

- la implementación propia de MT19937;
- `random`, que es exactamente el mismo algoritmo implementado en C dentro de CPython. Es reproducible a partir de una semilla y no sirve para criptografía;
- `secrets`, que es la interfaz al generador criptográfico del sistema operativo (`/dev/urandom` en Linux). No acepta semilla y no es reproducible, precisamente porque su propósito es que nadie pueda predecir su salida.

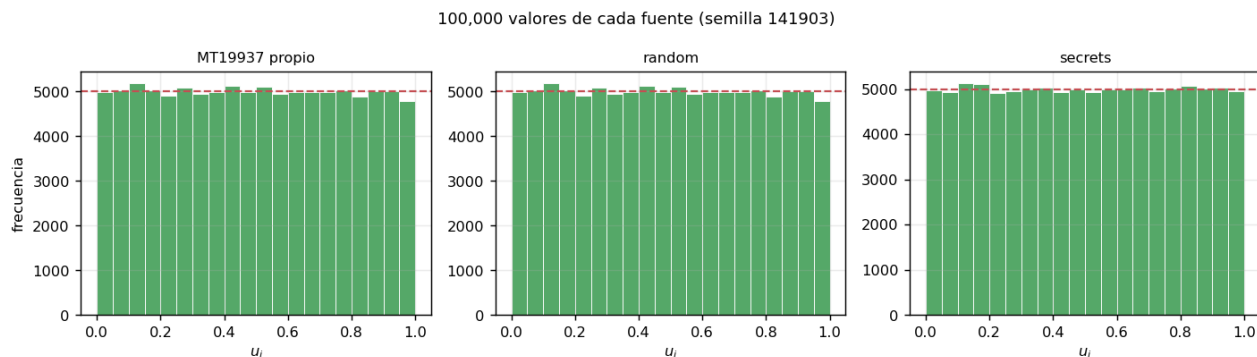


Figura 7: Histogramas de $N = 100,000$ valores de las tres fuentes.

Tabla 7: MT19937 propio frente a `random` y `secrets` ($N = 100,000$).

Fuente	Valores/s	X^2	p_{χ^2}	D	p_{KS}
MT19937 propio	459,593	11.71	0.2300	0.00381	0.1092
<code>random</code>	15,382,116	11.71	0.2300	0.00381	0.1092
<code>secrets</code>	1,174,510	9.34	0.4066	0.00205	0.7965

De la tabla 7 salen tres lecturas. La primera es que las tres fuentes son estadísticamente equivalentes: todos los p -valores de χ^2 y KS están muy por encima de 0,05, y entre la implementación propia y `random` los estadísticos coinciden *exactamente*, lo cual no es coincidencia sino la consecuencia de que son literalmente los mismos números. La segunda es que la diferencia de velocidad pasa del orden de magnitud, y eso es puro costo de Python interpretado contra C, porque el algoritmo es idéntico: sirve para recordar que “lento” casi nunca es propiedad del algoritmo sino de cómo está escrito. Y la tercera es que `secrets` sale más lento que `random` y encima no es reproducible, así que para simulación es la peor de las tres. Suena contraintuitivo que el generador más seguro sea el peor para esto, pero tiene sentido: en simulación uno quiere poder repetir el experimento con la misma semilla, y eso es justamente lo que un generador criptográfico está diseñado para impedir.

2.5. Blum Blum Shub

2.5.1. Fundamento matemático

BBS (Blum, Blum y Shub, 1986) es de una familia distinta: es un generador *criptográficamente seguro*, y su diseño no busca velocidad sino una garantía demostrable. Se toma $N = pq$ con p y q primos distintos tales que

$$p \equiv q \equiv 3 \pmod{4}, \quad (13)$$

lo que hace de N un *entero de Blum*, y se itera el cuadrado modular

$$x_{n+1} = x_n^2 \pmod{N}, \quad (14)$$

extrayendo en cada paso los j bits menos significativos de x_n , con $j = \lfloor \log_2 \log_2 N \rfloor$.

Por qué los primos tienen que ser congruentes con 3 módulo 4. Sea QR_N el conjunto de residuos cuadráticos módulo N . Cuando $p \equiv 3 \pmod{4}$, exactamente una de las dos raíces cuadradas de un residuo cuadrático módulo p es a su vez un residuo cuadrático; esa raíz se llama la raíz principal. Con la misma condición sobre q , el mapa $x \mapsto x^2$ resulta ser una *permutación* de QR_N . Esto es lo que garantiza que la órbita sea un ciclo puro, sin cola: no hay estados que se visiten una sola vez y el generador nunca se degrada como los cuadrados medios.

Seguridad. Predecir el siguiente bit de salida con ventaja no despreciable sobre el volado es tan difícil como decidir residuosidad cuadrática módulo N , problema que se cree tan difícil como factorizar N . Ésta es la diferencia cualitativa con todos los generadores anteriores: la seguridad no es una observación empírica, es una reducción. Sacar $j = \lfloor \log_2 \log_2 N \rfloor$ bits por paso, y no más, es parte del teorema: es la cantidad de bits que se puede extraer sin debilitar la reducción.

Un error que valió la pena cometer. Aquí hay algo que al principio parecía un bug del código y terminó siendo teoría: la condición $p \equiv q \equiv 3 \pmod{4}$ es necesaria pero **no suficiente** para tener buen período. El período de la órbita de (14) es

$$\text{ord}_{\lambda(\lambda(N))}(2), \quad (15)$$

el orden multiplicativo de 2 módulo $\lambda(\lambda(N))$, donde λ es la función de Carmichael. La razón es que $x_n = x_0^{2^n} \pmod{N}$, de modo que el exponente 2^n vive módulo $\lambda(N)$ y el período se reduce a preguntar cuándo 2^n se repite ahí.

En el primer intento usé los primos de Mersenne $p = 2^{31} - 1$ y $q = 2^{61} - 1$, que sí cumplen lo de 3 mód 4 y además son famosos, así que parecían una buena idea. El generador colapsó a un ciclo de unos 30 estados: de los 64 valores posibles de un bloque de 6 bits sólo aparecían 44, y las pruebas de uniformidad rechazaban H_0 por goleada. La causa es que $p - 1$ y $q - 1$ se factorizan en primos chiquitos, entonces $\lambda(\lambda(N))$ queda pequeño y el orden de 2 también. Ser primo no alcanza: importa cómo se factoriza el número de al lado.

La solución es exigir *primos seguros*, es decir $p = 2p' + 1$ con p' también primo, lo que obliga a $\lambda(N)$ a tener un factor primo grande. La implementación final usa

$$p = 4294967387 = 2 \cdot 2147483693 + 1, \quad q = 8589935363 = 2 \cdot 4294967681 + 1,$$

ambos primos seguros y congruentes con 3 mód 4. Con esos parámetros el generador pasa las cuatro pruebas.

2.5.2. Pseudocódigo

Algoritmo 2.4 | Blum Blum Shub

Entrada: primos seguros $p \equiv q \equiv 3 \pmod{4}$; semilla s coprime con $N = pq$ y distinta de 0 y 1; cantidad N_{out} de valores; ancho de salida $w = 32$ bits.

Salida: sucesión $u_1, \dots, u_{N_{\text{out}}}$ de valores en $[0, 1)$.

```

1  $N \leftarrow p \cdot q; \quad j \leftarrow \lfloor \log_2 \log_2 N \rfloor$ 
2 Verificar que  $\text{mcd}(s, N) = 1$ ; si no, la semilla es inválida.
3  $x \leftarrow s^2 \pmod{N}$  (entra al conjunto de residuos cuadráticos)
4 para  $i = 1, \dots, N_{\text{out}}$ :
    4.1  $v \leftarrow 0; \quad b \leftarrow 0$ 
    4.2 mientras  $b < w$ :
        4.2.1  $x \leftarrow x^2 \pmod{N}$ 
        4.2.2  $t \leftarrow \min(j, w - b)$  (no tomar más bits de los que faltan)
        4.2.3  $v \leftarrow (v \ll t) \mid (x \& (2^t - 1))$ 
        4.2.4  $b \leftarrow b + t$ 
    4.3  $u_i \leftarrow v/2^w$ 
5 devolver  $u_1, \dots, u_{N_{\text{out}}}$ 

```

2.5.3. Ficha comparativa

Tabla 8: Características de Blum Blum Shub.

Velocidad	Muy baja. Cada bloque de j bits cuesta una multiplicación modular sobre enteros de 66 bits, así que un solo valor de 32 bits requiere varias. Medido: 369,097 valores por segundo, unas diez veces menos que el LCG.
Período	Es $\text{ord}_{\lambda(\lambda(N))}(2)$ y depende finamente de p y q : con primos mal elegidos puede ser de unas decenas de estados, con primos seguros es astronómico.
Seguridad	Demostrable. Predecir la salida equivale a resolver el problema de la residuosidad cuadrática, que se cree tan difícil como factorizar N . Es el único de los cinco con una garantía de este tipo.
Casos de uso	Criptografía: generación de claves y <i>nonces</i> , cuando el costo por bit se puede tolerar. Nunca para simulación masiva.

2.5.4. Resultados de la implementación

Tabla 9: Pruebas de hipótesis para Blum Blum Shub sobre $N = 100,000$ valores. Se rechaza H_0 al 5 % si $p < 0,05$.

Prueba	Mide	Estadístico	p -valor	Decision
χ^2 ($k = 10$)	uniformidad	3.27	0.9527	no rechaza H_0
Kolmogorov-Smirnov	uniformidad	0.00132	0.9948	no rechaza H_0
Autocorrelacion lag-1	independencia	+0.00048	0.8786	no rechaza H_0
Rachas	independencia	49993	0.9596	no rechaza H_0

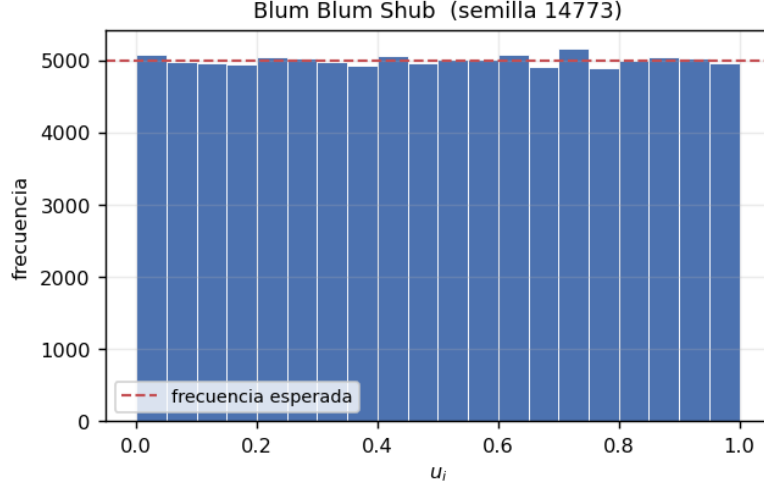


Figura 8: Histograma de $N = 100,000$ valores de Blum Blum Shub con semilla 14773.

Ya con los primos seguros, el generador se porta muy bien: $X^2 = 3,27$ con $p = 0,9527$ y $D = 0,00132$ con $p = 0,9948$ del lado de la uniformidad, y $r_1 = +0,00048$ con $p = 0,8786$ más $R = 49,993$ rachas con $p = 0,9596$ del lado de la independencia. Ninguna de las cuatro rechaza H_0 . Lo más interesante de esta tabla es lo que no se ve: estos mismos números, con los primos de Mersenne del primer intento, rechazaban H_0 en las cuatro pruebas. El código era idéntico, lo único que cambió fueron dos constantes. En criptografía los parámetros no son configuración, son parte del algoritmo.

2.6. RANDU

2.6.1. Fundamento matemático

RANDU es el LCG multiplicativo que IBM distribuyó en los años 60 dentro de su System/360 y que se usó durante más de una década en trabajo científico:

$$X_{n+1} = 65539 X_n \text{ mód } 2^{31}, \quad X_0 \text{ impar.} \quad (16)$$

La elección parece razonable a primera vista: $a = 65539 = 2^{16} + 3$ permite calcular la multiplicación con un desplazamiento y dos sumas, que en hardware de los años 60 era una ventaja real.

Período. Como $c = 0$, no aplica Hull–Dobell. Para un generador multiplicativo con $m = 2^{31}$, el estado nunca puede ser par si la semilla es impar (multiplicar impares da impar), así que la órbita vive en un subconjunto. El período máximo posible es $m/4 = 2^{29}$ y RANDU lo alcanza. Ya de entrada es un período cuatro veces menor que el de un LCG mixto con el mismo módulo, pero ése no es el problema serio.

El defecto fatal. El problema es algebraico y exacto. Escribiendo $a = 65539 = 2^{16} + 3$:

$$\begin{aligned} a^2 &= (2^{16} + 3)^2 = 2^{32} + 6 \cdot 2^{16} + 9 \\ &= 6(2^{16} + 3) - 9 + 2^{32} \\ &\equiv 6a - 9 \pmod{2^{31}}, \end{aligned} \quad (17)$$

donde el término 2^{32} desaparece porque es múltiplo de 2^{31} . Multiplicando la relación $a^2 \equiv 6a - 9$ por X_n se obtiene, para todo n ,

$$X_{n+2} = 6X_{n+1} - 9X_n \pmod{2^{31}} \quad (18)$$

Es decir, cada valor es una combinación lineal entera exacta de los dos anteriores. Dividiendo entre 2^{31} , las ternas normalizadas satisfacen

$$9u_n - 6u_{n+1} + u_{n+2} = k, \quad k \in \mathbb{Z}. \quad (19)$$

Cuántos planos. La ecuación (19) dice que toda terna consecutiva cae en alguno de los planos paralelos de vector normal $(9, -6, 1)$. Como cada $u \in [0, 1)$, la combinación $9u_n - 6u_{n+1} + u_{n+2}$ está acotada entre -6 y 10 , y sólo puede tomar los valores enteros en ese rango: exactamente 15 planos. La cota de Marsaglia (6) para $m = 2^{31}$ y $d = 3$ permitiría hasta $(6 \cdot 2^{31})^{1/3} \approx 2344$ planos, así que RANDU aprovecha menos del uno por ciento de la estructura que su módulo le permitiría. Esto es lo que Marsaglia bautizó como “los números aleatorios caen principalmente en los planos” [1].

2.6.2. Pseudocódigo

Algoritmo 2.5 | RANDU

Entrada: semilla impar X_0 ; cantidad N .

Salida: sucesión $u_1, \dots, u_N$ de valores en $[0, 1)$.

- 1 Verificar que X_0 sea impar; si es par, la órbita se degrada.
- 2 $X \leftarrow X_0 \pmod{2^{31}}$
- 3 **para** $i = 1, \dots, N$:
 - 3.1 $X \leftarrow 65539 \cdot X \pmod{2^{31}}$
 - 3.2 $u_i \leftarrow X/2^{31}$
- 4 **devolver** $u_1, \dots, u_N$

2.6.3. Ficha comparativa

Tabla 10: Características de RANDU.

Velocidad	Muy alta. Una multiplicación y un módulo por valor, y la multiplicación se puede hacer con desplazamientos. Medido: 3,207,784 valores por segundo, el más rápido de los cinco.
Período	2^{29} con semilla impar, la cuarta parte del módulo.
Seguridad	Nula, y además con una relación lineal explícita entre términos consecutivos que permite predecir la sucesión con sólo dos valores.
Casos de uso	Ninguno. Es el contraejemplo canónico de mal diseño y sólo se usa hoy con fines didácticos, como aquí.

2.6.4. Resultados de la implementación

Y aquí está el punto central de toda la Parte 1. RANDU **pasa las cuatro pruebas**: $X^2 = 8,80$ con $p = 0,4561$, $D = 0,00220$ con $p = 0,7185$, $r_1 = -0,00024$ con $p = 0,9390$ y $R = 50,136$ rachas

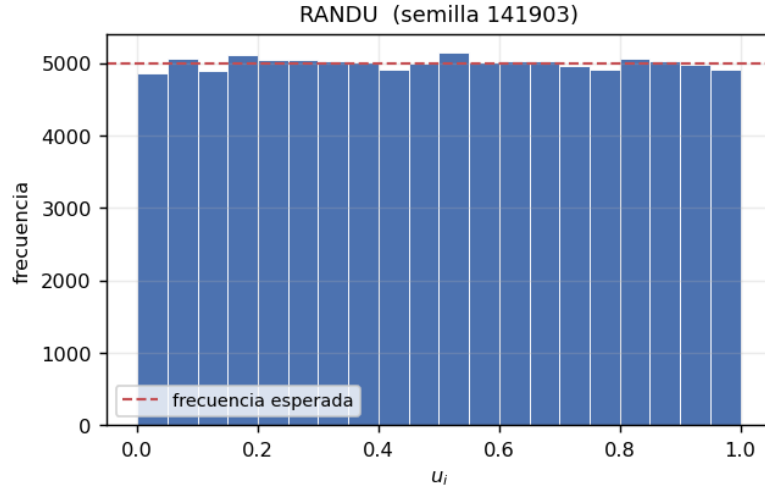


Figura 9: Histograma de $N = 100,000$ valores de RANDU con semilla 141903.

Tabla 11: Pruebas de hipotesis para RANDU sobre $N = 100,000$ valores. Se rechaza H_0 al 5 % si $p < 0,05$.

Prueba	Mide	Estadistico	p -valor	Decision
χ^2 ($k = 10$)	uniformidad	8.80	0.4561	no rechaza H_0
Kolmogorov–Smirnov	uniformidad	0.00220	0.7185	no rechaza H_0
Autocorrelacion lag-1	independencia	-0.00024	0.9390	no rechaza H_0
Rachas	independencia	50136	0.3932	no rechaza H_0

con $p = 0,3932$. Si uno se quedara con esta tabla, tendría que concluir que RANDU está al nivel del Mersenne Twister, lo cual sabemos que es falso.

Y no es que a las pruebas les haya faltado potencia estadística, ni que N sea chico. El problema es estructural: chi-cuadrado y KS miran un valor a la vez, la autocorrelación y las rachas miran parejas, y el defecto de RANDU vive en las *ternas*. Ninguna de las cuatro mira ternas, así que no hay forma de que lo encuentren. Subir N a mil millones tampoco serviría de nada; lo que hay que cambiar es la herramienta, no el tamaño de la muestra.

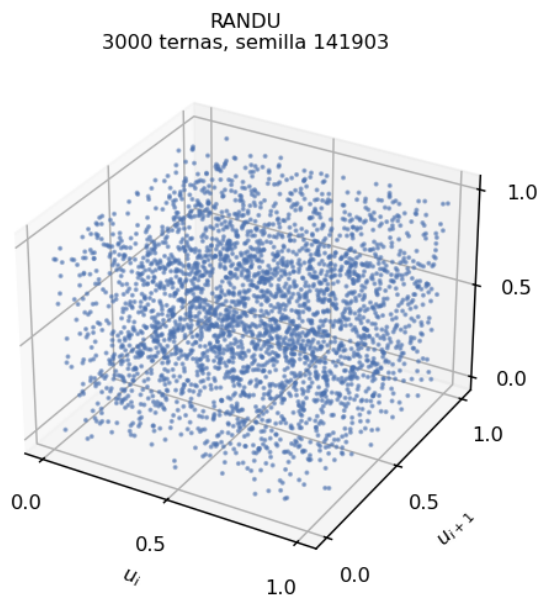


Figura 10: Ternas consecutivas de RANDU vistas desde un ángulo arbitrario. Comparar con las figuras 2 y 6: desde aquí los tres generadores se ven igual de aceptables.

La figura 10 está puesta ahí a propósito para engañar: mirada desde un ángulo cualquiera, la nube de RANDU se ve igual de llena que la del LCG o la del Mersenne Twister. La figura 11 es exactamente la misma nube, sólo que girada hasta quedar mirando en la dirección del vector normal $(9, -6, 1)$, y de repente todo el volumen se aplasta en 15 láminas. El panel de la derecha lo confirma sin lugar a dudas: la combinación de la ecuación (19) toma exactamente 15 valores enteros, con un residuo máximo de 0, o sea cero hasta donde la máquina puede medir. Esto no es una tendencia estadística que uno podría discutir, es una identidad algebraica.

De aquí salen dos cosas. La primera es que el defecto no aparece solo: hay que saber qué buscar, y quien dice dónde mirar es el álgebra de la ecuación (18), no los datos. Si uno se limita a graficar desde el ángulo por defecto, no encuentra nada. La segunda es que todo LCG tiene esta misma estructura, incluido el de la sección anterior; la diferencia es que uno bien parametrizado reparte sus ternas entre miles de planos pegaditos y RANDU las reparte entre 15 planos separados por huecos enormes. Y eso conecta con el período de una forma que vale la pena decir explícito: RANDU produce 2^{29} valores distintos, que no es poco, pero la calidad no se mide sólo por cuántos números salen antes de repetirse, sino por cómo se acomodan en el espacio donde uno los va a usar. En la Parte 2 se ve qué le pasa a un cálculo de Monte Carlo en tres dimensiones cuando lo alimenta este generador.

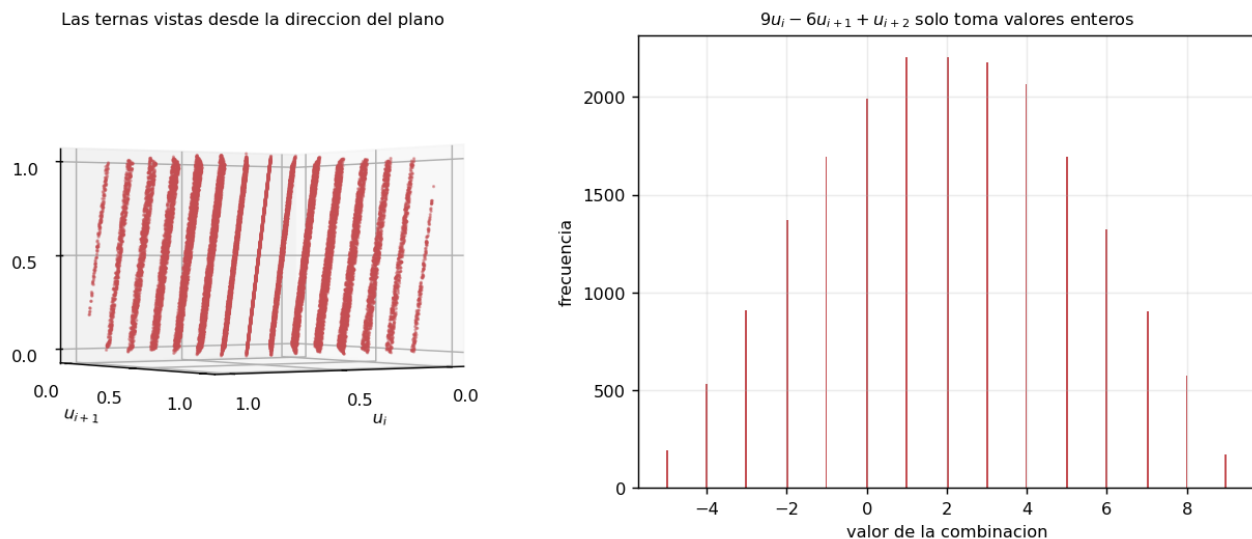


Figura 11: Izquierda: las mismas ternas vistas desde una dirección perpendicular al vector normal $(9, -6, 1)$; los 15 planos aparecen de canto. Derecha: la combinación $9u_i - 6u_{i+1} + u_{i+2}$ sólo toma 15 valores enteros distintos, con residuo máximo a entero de 0.

Tabla 12: Resumen comparativo. Las velocidades son las medidas en Python puro (tabla 13).

Algoritmo	Velocidad	Período	Seguridad	Casos de uso
LCG	Muy alta (3,045,229/s)	2^{32} por Hull–Dobell	Nula: el estado se recupera con pocas salidas	Simulación no crítica, videojuegos, sistemas embebidos
Cuadrados medios	Alta (1,655,135/s)	Sin garantía; degenera. Aquí 8,864 pasos	Nula	Sólo interés histórico y didáctico
Mersenne Twister	Media (982,384/s)	$2^{19937} - 1$	Nula: 624 salidas bastan para reconstruir el estado	Estándar en simulación científica: Python, R, MATLAB, NumPy
Blum Shub	Muy baja (369,097/s)	$\text{ord}_{\lambda(\lambda(N))}(2)$; exige primos seguros	Demostrable: equivale a factorizar N	Criptografía: claves y <i>nonces</i>
RANDU	Muy alta (3,207,784/s)	2^{29}	Nula, con relación lineal explícita entre ternas	Ninguno. Contraejemplo canónico

Tabla 13: Velocidad medida en Python puro, 50 000 valores por generador.

Generador	Valores por segundo	Costo relativo
LCG	3,045,229	1.1x
Cuadrados medios	1,655,135	1.9x
Mersenne Twister	982,384	3.3x
Blum Blum Shub	369,097	8.7x
RANDU	3,207,784	1.0x

2.7. Comparación global de los cinco generadores

Ojo con lo que mide la tabla 13: es el costo en Python interpretado, no la velocidad intrínseca de cada algoritmo. El orden relativo sí dice algo, porque refleja cuántas operaciones aritméticas necesita cada generador para entregar un valor. RANDU y el LCG son los más baratos con una multiplicación y un módulo; el Mersenne Twister paga el mantenimiento de su estado de 624 palabras; y Blum Blum Shub va casi diez veces más lento que el LCG porque necesita varias multiplicaciones modulares de 66 bits para armar un solo valor de 32. Los números absolutos, en cambio, no se pueden comparar con una implementación en C: la misma tabla 7 muestra que el mismo algoritmo corre más de 20 veces más rápido una vez compilado.

Si hay que resumir la Parte 1 en una frase, los cinco generadores caen en tres grupos. Cuadrados medios falla de la forma más obvia que existe y lo agarra cualquier prueba. LCG, Mersenne Twister y Blum Blum Shub pasan todo, cada uno con su propio trato entre velocidad, período y seguridad. Y RANDU también pasa todo, pero por la razón equivocada: tiene un defecto grave que las pruebas que se le aplicaron simplemente no pueden ver.

3. Parte 2: Métodos de Monte Carlo

La idea de Monte Carlo es cambiar un cálculo exacto por una estimación: si la cantidad que uno busca se puede escribir como un valor esperado, entonces se muestrea y se promedia, y ya. La Ley de los Grandes Números garantiza que ese promedio llega a donde tiene que llegar, y el Teorema del Límite Central dice qué tan rápido y con cuánta incertidumbre. Toda estimación numérica de esta parte viene acompañada de su intervalo de confianza del 95 %.

3.1. Fundamento: el estimador y su error

3.1.1. La integral como valor esperado y el insesgamiento

Sea f integrable en $[a, b]$ y sea

$$I = \int_a^b f(x) dx. \quad (20)$$

Tomemos $U \sim \text{Unif}(a, b)$, cuya densidad es

$$p_U(x) = \begin{cases} \frac{1}{b-a} & x \in [a, b], \\ 0 & \text{en otro caso.} \end{cases} \quad (21)$$

Por definición de valor esperado de una función de una variable aleatoria continua,

$$\mathbb{E}[f(U)] = \int_{-\infty}^{\infty} f(x) p_U(x) dx = \int_a^b f(x) \frac{1}{b-a} dx = \frac{I}{b-a}, \quad (22)$$

y despejando se obtiene la identidad que sostiene todo el método:

$$\boxed{I = (b-a) \mathbb{E}[f(U)]} \quad (23)$$

La integral es el ancho del intervalo por la altura promedio de f , o sea el área de un rectángulo. Eso es todo lo que hay detrás del método, y conviene tenerlo a mano antes de llenar la página de sumatorias.

Ahora bien, un valor esperado se puede estimar con un promedio muestral. Sean $U_1, \dots, U_N$ independientes e idénticamente distribuidas como $\text{Unif}(a, b)$, y definamos

$$\hat{I}_N = \frac{b-a}{N} \sum_{i=1}^N f(U_i). \quad (24)$$

Para ver que es insesgado basta con la linealidad del valor esperado, que no requiere independencia:

$$\mathbb{E}[\hat{I}_N] = \frac{b-a}{N} \sum_{i=1}^N \mathbb{E}[f(U_i)] = \frac{b-a}{N} \cdot N \cdot \frac{I}{b-a} = I, \quad (25)$$

donde lo único que se usó es que todas las U_i tienen la misma distribución, así que cada término aporta el mismo valor esperado $I/(b-a)$. O sea que $\mathbb{E}[\hat{I}_N] = I$ para cualquier $N \geq 1$, y eso incluye $N = 1$: el estimador no tiene sesgo ni con una sola muestra. Vale la pena detenerse en esto porque es fácil malinterpretarlo. Lo que mejora cuando crece N no es el centro de la distribución del estimador, que ya estaba en el lugar correcto, sino qué tan apretada está alrededor de ese centro.

3.1.2. Varianza y la tasa de convergencia

Aquí sí hace falta la independencia. Escribamos $\sigma^2 = \text{Var}(f(U))$, que suponemos finita. Como la varianza de una suma de variables independientes es la suma de las varianzas, y como $\text{Var}(cX) = c^2 \text{Var}(X)$,

$$\begin{aligned} \text{Var}(\hat{I}_N) &= \text{Var}\left(\frac{b-a}{N} \sum_{i=1}^N f(U_i)\right) \\ &= \left(\frac{b-a}{N}\right)^2 \sum_{i=1}^N \text{Var}(f(U_i)) \\ &= \frac{(b-a)^2}{N^2} \cdot N\sigma^2 = \frac{(b-a)^2\sigma^2}{N}. \end{aligned} \quad (26)$$

El error estándar es la raíz de esa varianza:

$$\text{ee}(\hat{I}_N) = \frac{(b-a)\sigma}{\sqrt{N}} = \mathcal{O}(N^{-1/2}). \quad (27)$$

Para ganar un dígito de precisión hay que multiplicar el trabajo por cien, y para duplicarla hay que cuadruplicarlo. Es una tasa francamente lenta, sobre todo comparada con una regla de Simpson en una dimensión, que converge como N^{-4} .

Por qué la tasa no depende de la dimensión. Repitamos el argumento en un dominio $D \subset \mathbb{R}^d$ de volumen finito $|D|$. Con $\mathbf{U}$ uniforme en D se tiene $I = |D| \mathbb{E}[f(\mathbf{U})]$ por el mismo cálculo de arriba, y el estimador $\hat{I}_N = \frac{|D|}{N} \sum_i f(\mathbf{U}_i)$ tiene varianza $|D|^2 \sigma^2 / N$ con $\sigma^2 = \text{Var}(f(\mathbf{U}))$. Lo importante aquí es notar que en ningún paso del cálculo apareció d por ningún lado. Y la razón de fondo es bonita: al promedio muestral le da igual en qué espacio viven los puntos, porque lo único que ve son N números $f(\mathbf{U}_i)$ independientes y con la misma varianza. Que esos números vengan de dos o de veinte dimensiones no cambia nada de la cuenta. La dimensión se cuela sólo en el valor de σ y en $|D|$, que son constantes respecto a N , pero nunca toca el exponente $-1/2$.

Y aquí es donde esto se vuelve útil de verdad, comparándolo con un método determinista. Una cuadratura que parte cada eje en m pedazos gasta $N = m^d$ puntos, así que si su error es $\mathcal{O}(m^{-k})$ en una dimensión, en d dimensiones queda

$$\mathcal{O}(m^{-k}) = \mathcal{O}(N^{-k/d}), \quad (28)$$

que empeora conforme d crece. Monte Carlo gana en cuanto $d > 2k$. Esto se verifica numéricamente en la sección 3.3.

3.1.3. Intervalo de confianza del 95 %

Por el Teorema del Límite Central, el promedio de variables i.i.d. con varianza finita es asintóticamente normal. Aplicado a (24),

$$\frac{\hat{I}_N - I}{(b-a)\sigma/\sqrt{N}} \xrightarrow{d} \mathcal{N}(0, 1). \quad (29)$$

Como σ no se conoce, se reemplaza por su estimador muestral

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (f(U_i) - \bar{f})^2, \quad \bar{f} = \frac{1}{N} \sum_{i=1}^N f(U_i), \quad (30)$$

lo cual es legítimo por el teorema de Slutsky, ya que $s \rightarrow \sigma$ en probabilidad. Con $z_{0,975} = 1,96$ el intervalo de confianza del 95 % queda

$$\boxed{\hat{I}_N \pm 1,96 \frac{(b-a)s}{\sqrt{N}}} \quad (31)$$

Conviene tener clara la interpretación, porque casi siempre se dice mal. El intervalo no significa “hay 95 % de probabilidad de que I esté aquí adentro”; I es un número fijo, no tiene probabilidad. Lo que significa es que si uno repitiera todo el experimento muchas veces con semillas distintas, como el 95 % de los intervalos que va construyendo contendrían el valor verdadero. Eso no se queda en la teoría: en la sección 3.5 se repite el experimento 100 veces y la cobertura observada con un buen generador sale en 97 %.

Un caso particular que se usa mucho más adelante: cuando lo que se estima es una proporción, como en los problemas de volumen de las siguientes secciones, el integrando es un indicador que sólo vale 0 o 1. Ahí $s^2 = \hat{p}(1 - \hat{p})$ y el intervalo se reduce a $\hat{p} \pm 1,96 \sqrt{\hat{p}(1 - \hat{p})/N}$. Esa fórmula tiene una trampa: si no cae ningún acierto, $\hat{p} = 0$ y el error estándar estimado da cero, o sea que el intervalo sería un punto y estaría diciendo que hay certeza absoluta, lo cual es absurdo. Para ese caso existe la regla de tres, que da como cota superior al 95 % el valor $3/N$.

3.2. Calentamiento: integrales en una dimensión

Se estimaron las dos integrales

$$(a) \int_0^1 \sin(\pi x) dx = \frac{2}{\pi} = 0,636620, \quad (b) \int_0^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx = \Phi(2) - \Phi(0) = 0,477250, \quad (32)$$

con $N = 1,000,000$ muestras y tres fuentes de aleatoriedad: el módulo **random** de Python, el LCG propio y el MT19937 propio.

Tabla 14: Estimaciones con $N = 1,000,000$ muestras e intervalo de confianza del 95 %.

Integral	Fuente	Estimacion	IC 95 %	Error	Cubre
(a) seno	random	0,636804	[0,636201, 0,637408]	$1,8 \times 10^{-4}$	si
(a) seno	LCG	0,636428	[0,635825, 0,637032]	$1,9 \times 10^{-4}$	si
(a) seno	MT19937	0,637334	[0,636731, 0,637937]	$7,1 \times 10^{-4}$	no
(b) normal	random	0,477583	[0,477132, 0,478035]	$3,3 \times 10^{-4}$	si
(b) normal	LCG	0,477697	[0,477245, 0,478148]	$4,5 \times 10^{-4}$	si
(b) normal	MT19937	0,477344	[0,476893, 0,477795]	$9,4 \times 10^{-5}$	si

Lectura de los resultados. Las seis estimaciones quedan a menos de 10^{-3} del valor exacto, con errores del orden de 10^{-4} , que es exactamente lo que predice la ecuación (27): con $N = 1,000,000$ y $\sigma \approx 0,3$ el error estándar esperado anda por 3×10^{-4} . Hasta ahí, todo en orden.

Ahora, cinco de los seis intervalos contienen el valor verdadero y el sexto no. El que falla es el de MT19937 en la integral (a), que se queda a unos dos errores estándar. Lo dejo reportado tal cual porque no es un error del método ni del generador: un intervalo del 95 % está diseñado para fallar una de cada veinte veces, y con seis intervalos la probabilidad de que al menos uno falle es de más o menos 26 %. O sea que era bastante esperable que pasara. Si nunca fallara ninguno, el intervalo estaría mal calculado y sería demasiado ancho. De todas formas, la verificación en serio de esto está en la sección 3.5, donde se repite el experimento 100 veces y se cuenta la cobertura real.

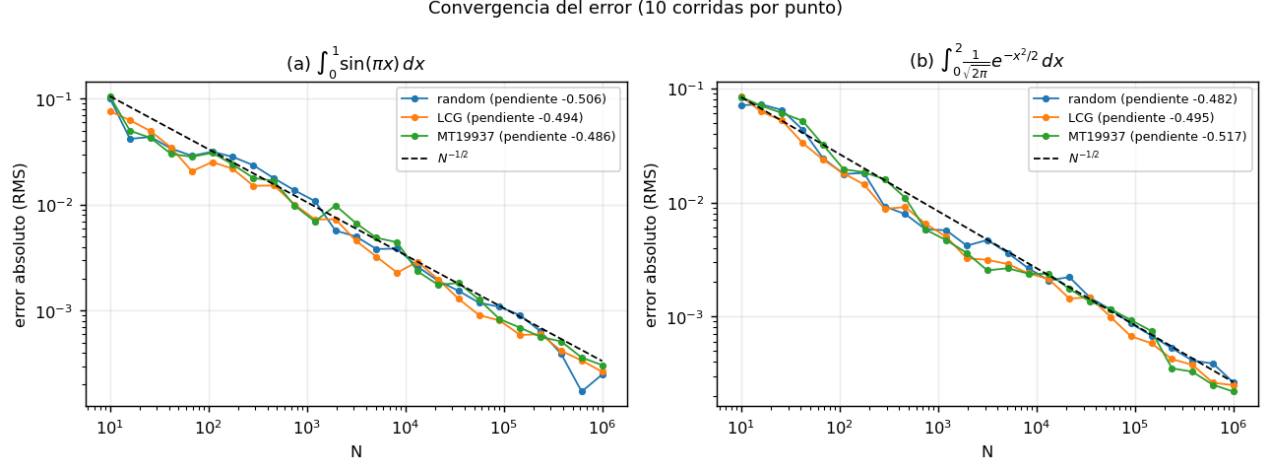


Figura 12: Error absoluto contra N en escala log–log, promediado sobre 10 corridas independientes por punto. La recta punteada es la referencia $N^{-1/2}$.

Tabla 15: Pendiente ajustada al error RMS en escala log–log. El valor teórico es $-0,5$.

Fuente	Integral (a)	Integral (b)
random	-0.506	-0.482
LCG	-0.494	-0.495
MT19937	-0.486	-0.517

Estudio de convergencia. La figura 12 barre N desde 10 hasta 10^6 . Una sola corrida sale utilísima para nada, porque el error salta muchísimo de un N al siguiente, así que cada punto de la gráfica es el error cuadrático medio sobre 10 corridas independientes con semillas distintas. Con eso, las seis rectas ajustadas dan pendientes entre $-0,48$ y $-0,52$ contra el $-0,5$ teórico, o sea que la tasa $\mathcal{O}(N^{-1/2})$ que se derivó arriba se cumple a lo largo de cinco órdenes de magnitud en N , desde 10^1 hasta 10^6 . Para la integral (a) con MT19937 la pendiente sale -0.486 y para la (b), -0.517 .

Las diferencias respecto a $-0,5$ son de unas dos centésimas y no hay que leerles nada raro: con 10 corridas, el propio error RMS trae una incertidumbre relativa cercana al 20 %, que es lo que se ve como temblor en la parte izquierda de la gráfica, donde N es chico. Ninguna de las tres fuentes da señales de salirse de la tasa teórica.

Comparación entre generadores. Este resultado hay que subrayarlo porque a primera vista parece contradecir toda la Parte 1: para estas dos integrales, el LCG propio funciona igual de bien que **random**. ¿Y entonces para qué tanto escándalo con la calidad de los generadores?

La explicación es que los dos integrandos son funciones suaves de *una* sola variable, así que el estimador nada más usa la distribución marginal de cada u_i por separado, y en eso los tres generadores son indistinguibles. Los defectos de correlación entre valores consecutivos, que es justo donde se rompen los generadores malos, en este problema ni se activan. Moraleja provisional: un generador mediocre puede funcionar perfecto mientras el problema no toque su punto débil. La sección 3.5 arma a propósito el caso contrario.

3.3. Alta dimensión y la maldición de la dimensionalidad

El volumen de la bola unitaria en d dimensiones es

$$V_d = \int_{[-1,1]^d} \mathbf{1}[x_1^2 + \dots + x_d^2 \leq 1] d\mathbf{x} = \frac{\pi^{d/2}}{\Gamma(\frac{d}{2} + 1)}. \quad (33)$$

La fórmula cerrada se usa sólo como verdad de referencia. El estimador muestrea puntos uniformes en el cubo $[-1,1]^d$ y cuenta la fracción $\hat{p}$ que cae dentro de la bola, de modo que $\hat{V}_d = 2^d \hat{p}$, con el intervalo de confianza para proporciones de la sección anterior.

Tabla 16: Volumen de la bola unitaria estimado con $N = 200,000$ puntos uniformes en $[-1,1]^d$.

d	V_d exacto	Aciertos	Estimacion	IC 95 %	Error
2	3.1416	157,047	3.1409	[3,1337, 3,1481]	0.0007
5	5.2638	32,862	5.2579	[5,2060, 5,3099]	0.0059
10	2.5502	490	2.5088	[2,2869, 2,7307]	0.0414
20	0.0258	0	0.0000	[0, 15,7286]	0.0258

El deterioro se ve clarísimo y va en dos etapas. Hasta $d = 5$ la estimación sale excelente y uno diría que no hay problema. En $d = 10$ el error relativo ya es de casi 2 % a pesar de estar usando $N = 200,000$ puntos, y la razón se lee en la columna de aciertos: de todos esos puntos, sólo 490 cayeron dentro de la bola. Y en $d = 20$ pasa lo que tenía que pasar: el estimador devuelve exactamente cero, porque de los 200,000 puntos, 0 cayeron dentro. Ahí el intervalo normal se degenera, así que hay que reportar la cota de la regla de tres, $\hat{V}_{20} \leq 15,7286$. Técnicamente esa cota sí contiene al valor verdadero $V_{20} = 0,0258$, pero es tan ancha que resulta inútil: es unas 600 veces más grande que la cantidad que se quería medir, así que no descarta prácticamente ningún valor.

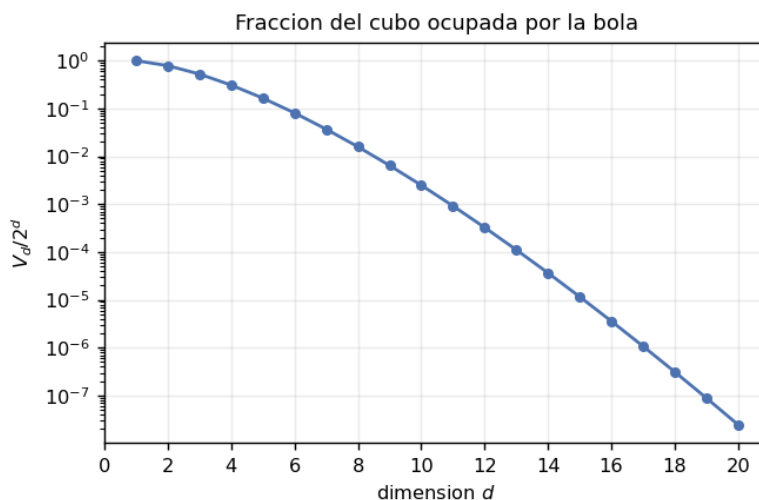


Figura 13: Fracción del volumen del cubo $[-1,1]^d$ que ocupa la bola unitaria, en escala logarítmica.

Lo que está pasando geoméricamente. La figura 13 explica el porqué de todo esto. La razón $V_d/2^d$ se desploma de manera superexponencial: vale $2,49 \times 10^{-3}$ en $d = 10$ y $2,46 \times 10^{-8}$ en $d = 20$. En veinte dimensiones la bola inscrita ocupa una parte en cuarenta millones del cubo que la contiene, lo cual es bastante difícil de creer viniendo de la intuición de dos o tres dimensiones, donde la bola se come más de la mitad del cuadrado o del cubo.

¿A dónde se fue todo el volumen? A las esquinas. Hay 2^{20} de ellas, y mientras la distancia del centro a una cara sigue siendo 1, la distancia a una esquina es $\sqrt{d}$. Entonces el cubo en dimensión alta no se parece en nada a un cubo de juguete: es una figura puntiaguda, con casi todo su contenido lejos del centro.

Para el diseño del estimador esto es letal. El muestreo uniforme en el cubo está gastando prácticamente todos sus puntos en una región que no aporta nada al resultado: para pescar *un solo* acierto en $d = 20$ harían falta como 40,631,628 puntos. El estimador no está roto, está mal escogido para el problema, y la salida no es subir N como bruto sino cambiar la distribución de muestreo por una que ponga los puntos donde está la masa, que es precisamente la idea del muestreo por importancia.

Y un matiz importante para no contradecir lo que se derivó antes: la tasa sigue siendo $\mathcal{O}(N^{-1/2})$ en cualquier dimensión. Lo que empeora es la *constante* σ , porque para un indicador con probabilidad p chiquita el error relativo es $\sqrt{(1-p)/(pN)}$, que crece como $1/\sqrt{p}$. La dimensión no ataca al exponente, le pega a la constante.

Tabla 17: Monte Carlo contra cuadratura en rejilla con el mismo presupuesto de puntos, y puntos que exigiría una rejilla de $m = 10$ subdivisiones por eje.

d	m posible	Puntos usados	Error rejilla	Error MC	Puntos si $m = 10$
2	316	99,856	0.0012	0.0007	10^2
5	10	100,000	0.3793	0.0059	10^5
10	3	59,049	0.9355	0.0414	10^{10}
20	1	1	1048575.9742	0.0258	10^{20}

Comparación contra la rejilla. Con el mismo presupuesto de 100,000 puntos, la cuadratura determinista evalúa el integrando en el centro de cada celda de una rejilla de m subdivisiones por eje, con $m = \lfloor 100,000^{1/d} \rfloor$. En $d = 2$ la rejilla gana, como era de esperarse: con $m = 316$ subdivisiones el error es comparable al de Monte Carlo. El problema es que ese presupuesto hay que repartirlo entre todos los ejes, así que m se viene abajo apenas crece d : en $d = 5$ ya sólo alcanza para $m = 10$, en $d = 10$ para $m = 3$ y en $d = 20$ para $m = 1$. Ese último caso es casi cómico. La rejilla de un solo punto evalúa el integrando en el centro del cubo, que sí está dentro de la bola, concluye alegremente que *todo* el cubo está dentro y devuelve $\hat{V}_{20} = 2^{20} \approx 1.05 \times 10^6$ cuando el valor real es 0.0258. El resultado es unas 4×10^7 veces más grande que el verdadero y, peor todavía, lo entrega sin ninguna señal de alarma: un método determinista no trae intervalo de confianza que avise que algo anda mal.

La última columna de la tabla 17 es el argumento en su versión más limpia. Para mantener una resolución fija de $m = 10$ subdivisiones por eje hacen falta 10^d puntos: son 100 en dos dimensiones, 10^{10} en diez y 10^{20} en veinte, que es más de lo que cualquier computadora va a evaluar en lo que a uno le queda de vida. Eso es la maldición de la dimensionalidad: cubrir el espacio con resolución fija cuesta exponencialmente en d , mientras que a Monte Carlo la dimensión sólo lo toca a través de σ .

3.4. Reducción de varianza

Como el error estándar es $(b-a)\sigma/\sqrt{N}$, hay exactamente dos maneras de reducirlo: subir N , que es caro y sólo paga $\sqrt{N}$, o bajar σ , que puede pagar mucho más. Se implementaron dos técnicas y se aplicaron a las dos integrales de la sección anterior, siempre con el mismo número de evaluaciones de f para que la comparación sea justa.

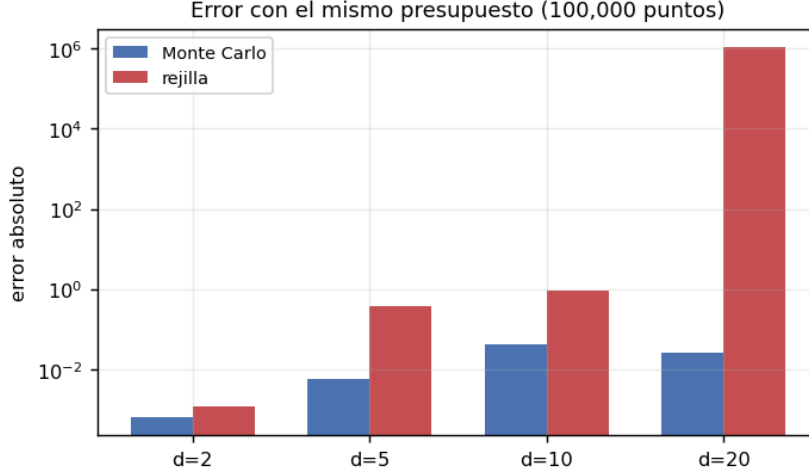


Figura 14: Error absoluto de los dos métodos con el mismo presupuesto de puntos, en escala logarítmica.

3.4.1. Variables antitéticas

En lugar de promediar N evaluaciones independientes, se generan $N/2$ parejas $(U, 1 - U)$, que tienen la misma distribución marginal pero están negativamente relacionadas, y se promedia dentro de cada pareja:

$$W = \frac{f(U) + f(1 - U)}{2}, \quad \hat{I}^{\text{ant}} = \frac{b - a}{N/2} \sum_{j=1}^{N/2} W_j. \quad (34)$$

Como U y $1 - U$ son ambas uniformes, el estimador sigue siendo insesgado. Su varianza es

$$\text{Var}(W) = \frac{1}{4} \left(\text{Var}(f(U)) + \text{Var}(f(1 - U)) + 2 \text{Cov}(f(U), f(1 - U)) \right) = \frac{\sigma^2(1 + \rho)}{2}, \quad (35)$$

donde ρ es la correlación entre $f(U)$ y $f(1 - U)$. Con el mismo presupuesto de N evaluaciones se usan $N/2$ parejas, así que

$$\text{Var}(\hat{I}^{\text{ant}}) = \frac{2}{N} \cdot \frac{\sigma^2(1 + \rho)}{2} = \frac{\sigma^2(1 + \rho)}{N}, \quad \text{factor de reducción} = \frac{1}{1 + \rho}. \quad (36)$$

La conclusión no puede ser más clara: la técnica ayuda si y sólo si $\rho < 0$, o sea si f es monótona o al menos se comporta al revés en los dos extremos del intervalo. Si $\rho > 0$, no es que no ayude, es que empeora. Esto va a importar en un momento.

3.4.2. Variables de control

Se busca una función h cuyo valor esperado $\mathbb{E}[h(U)]$ se conozca exactamente y que esté muy correlacionada con f . Se define

$$Y = f(U) - c(h(U) - \mathbb{E}[h(U)]). \quad (37)$$

Como el término restado tiene valor esperado cero, $\mathbb{E}[Y] = \mathbb{E}[f(U)]$ para cualquier constante c : el estimador es insesgado sin importar qué tan buena sea la elección de h . Lo que sí depende de c es la varianza,

$$\text{Var}(Y) = \sigma^2 - 2c \text{Cov}(f, h) + c^2 \text{Var}(h), \quad (38)$$

que es una parábola en c y se minimiza en

$$c^* = \frac{\text{Cov}(f, h)}{\text{Var}(h)}, \quad \text{con} \quad \text{Var}(Y)|_{c^*} = \sigma^2(1 - \rho_{f,h}^2), \quad (39)$$

donde $\rho_{f,h}$ es la correlación entre $f(U)$ y $h(U)$. El factor de reducción queda entonces en $1/(1 - \rho^2)$, que crece sin límite conforme la correlación se acerca a ± 1 . Lo que se gana es toda la parte de f que h alcanza a explicar: sólo queda por estimar el residuo. El coeficiente c^* se estimó con una muestra piloto aparte de 20 000 puntos, para no usar los mismos datos dos veces.

Los controles elegidos fueron $h(x) = x(1 - x)$ para la integral (a), con $\mathbb{E}[h] = 1/6$ sobre $\text{Unif}(0, 1)$, y $h(x) = x^2$ para la (b), con $\mathbb{E}[h] = 4/3$ sobre $\text{Unif}(0, 2)$. Ambos son polinomios cuya integral es elemental, que es justamente lo que se necesita: un control sirve sólo si su valor esperado se conoce de antemano.

3.4.3. Resultados

Tabla 18: Reduccion de varianza con $N = 200,000$ evaluaciones de f por estimador, sobre la integral (a) del seno y la (b) de la densidad normal. El factor compara la varianza del estimador contra Monte Carlo simple, y el speedup efectivo es el numero de muestras simples que harian falta para igualar esa precision.

Integral	Tecnica	Estimacion	IC 95 %	Varianza	Factor	Speedup
(a)	simple	0,636179	[0,634830, 0,637528]	$4,74 \times 10^{-7}$	1.00x	$2,0 \times 10^5$
	antiteticas	0,635877	[0,633970, 0,637785]	$9,47 \times 10^{-7}$	0.50x	$1,0 \times 10^5$
	control	0,636603	[0,636527, 0,636678]	$1,49 \times 10^{-9}$	318.55x	$6,4 \times 10^7$
(b)	simple	0,477290	[0,476280, 0,478299]	$2,65 \times 10^{-7}$	1.00x	$2,0 \times 10^5$
	antiteticas	0,477234	[0,477181, 0,477287]	$7,31 \times 10^{-10}$	363.07x	$7,3 \times 10^7$
	control	0,477307	[0,477073, 0,477542]	$1,43 \times 10^{-8}$	18.54x	$3,7 \times 10^6$

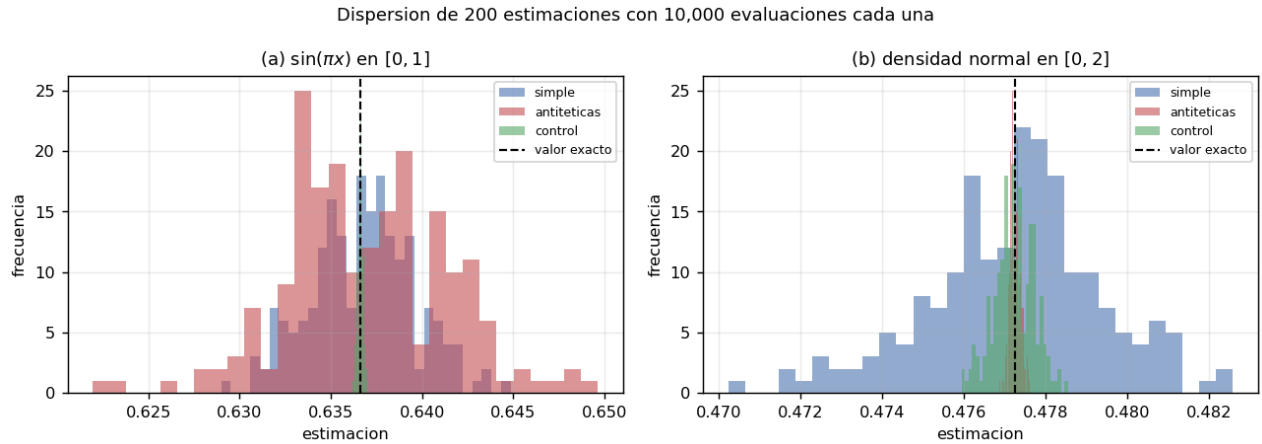


Figura 15: Dispersión de 200 estimaciones independientes con 10,000 evaluaciones cada una. Cuanto más angosto el histograma, menor la varianza del estimador.

Los cuatro resultados de la tabla 18 le pegan a las fórmulas (36) y (39) casi al decimal, sin haber acomodado nada:

- **Antitéticas en la integral (b): 363.07x.** La densidad normal es estrictamente decreciente en $[0, 2]$, así que $f(U)$ y $f(2-U)$ están fuertemente correlacionadas de forma negativa. La reducción es de más de dos órdenes de magnitud y sale gratis: no requiere información adicional sobre el integrando, sólo saber que es monótono.
- **Antitéticas en la integral (a): 0.50x, es decir, empeora.** Aquí está la respuesta a la pregunta de si la técnica siempre ayuda. La función $\sin(\pi x)$ es simétrica respecto a $x = 1/2$, de modo que $f(1-U) = f(U)$ *exactamente* y por lo tanto $\rho = +1$. La fórmula (36) predice un factor de $1/(1+1) = 0,5$, y lo medido es 0.50x. El estimador gasta dos evaluaciones para obtener la información de una: la pareja antitética es una copia idéntica.
- **Control en la integral (a): 318.55x.** La correlación medida entre $\sin(\pi x)$ y $x(1-x)$ es $\rho = 0,9984$, porque ambas son campanas simétricas que se anulan en los extremos. La fórmula predice $1/(1-0,9984^2) \approx 313$ y lo medido es 318.55x.
- **Control en la integral (b): 18.54x.** La correlación con x^2 es $-0,9731$ y la predicción $1/(1-\rho^2) \approx 18,8$ coincide con lo medido.

Qué significa eso en muestras. Un factor de reducción F quiere decir que la técnica llega, con N evaluaciones, a la misma precisión que Monte Carlo simple alcanzaría con $F \cdot N$. Ése es el speedup efectivo y aparece en la última columna de la tabla 18, expresado directamente en muestras simples equivalentes. Para el control en la integral (a), igualar por fuerza bruta la precisión que se obtuvo con $N = 200,000$ muestras pediría alrededor de 6×10^7 muestras simples. Ahí está el argumento económico de toda la sección: media hora pensando en un buen control vale más que dejar la computadora corriendo toda la noche.

¿Entonces siempre conviene usarlas? No, y cada técnica falla por un motivo distinto. Las antitéticas dependen del *signo* de ρ : ayudan con integrandos monótonos y estorban con integrandos simétricos, que es exactamente lo que se vio arriba. Las variables de control dependen de la *magnitud* de ρ y además piden conocer $\mathbb{E}[h]$ de antemano; si uno elige un control con correlación cercana a cero, el factor $1/(1-\rho^2)$ se va a 1 y no se gana nada, aunque al menos tampoco se pierde el insesgamiento. En los dos casos lo que manda es la forma del integrando, no la técnica, así que el orden correcto de trabajo es mirar primero cómo es f y después decidir.

3.5. Cuando el generador arruina el resultado: el caso RANDU

En la Parte 1 quedó demostrado que las ternas consecutivas de RANDU caen en 15 planos del cubo $[0, 1]^3$, y también que las cuatro pruebas de hipótesis no detectan nada. Falta ver qué le hace eso a un cálculo real. Se estimaron dos volúmenes en tres dimensiones, ambos con ternas consecutivas (u_i, u_{i+1}, u_{i+2}) y $N = 2,000,000$ ternas por estimación:

$$(i) \int_{[0,1]^3} \mathbf{1}[x^2 + y^2 + z^2 \leq 1] dx dy dz = \frac{\pi}{6} = 0,523599, \quad (40)$$

$$(ii) \int_{[0,1]^3} \mathbf{1}[x^2 + y^2 + z^2 \leq r^2] dx dy dz = \frac{\pi r^3}{6} = 0,000524, \quad r = 0,1. \quad (41)$$

El primero es el octante de la bola unitaria; el segundo es una bola pequeña pegada al origen. Los dos se estimaron con RANDU y con el Mersenne Twister.

Tabla 19: Dos estimaciones en tres dimensiones con $N = 2,000,000$ ternas consecutivas de cada generador. La ultima columna es la fraccion de los $K = 100$ intervalos que cubren el valor real.

Region	Fuente	Estimacion	IC 95 %	Sesgo relativo	$ z $	Cobertura
Bola unitaria	RANDU	0,523444	[0,522752, 0,524137]	-0,03 %	0,4	96 %
Bola unitaria	Mersenne Twister	0,524037	[0,523345, 0,524729]	+0,08 %	1,2	97 %
Bola $r = 0,1$	RANDU	0,000654	[0,000619, 0,000689]	+24,90 %	7,2	87 %
Bola $r = 0,1$	Mersenne Twister	0,000532	[0,000500, 0,000564]	+1,60 %	0,5	93 %

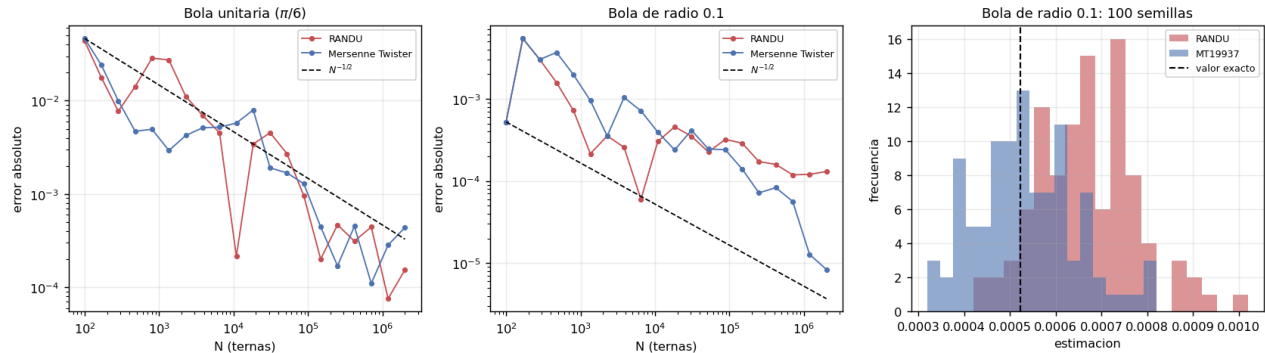


Figura 16: Izquierda y centro: error absoluto contra el número de ternas, en escala log-log. Derecha: dispersión de las estimaciones de la bola pequeña sobre 100 semillas distintas.

Lo que pasó con la región grande. Con el octante de la bola unitaria, RANDU y el Mersenne Twister dan prácticamente lo mismo: 0,523444 contra 0,524037, los dos a menos de dos errores estándar del valor exacto y los dos con intervalos que lo cubren. En el panel izquierdo de la figura 16 las dos curvas de error bajan pegaditas a la referencia $N^{-1/2}$. Si el análisis se hubiera quedado aquí, la conclusión habría sido que RANDU funciona sin problema, que es justo lo que le pasó a mucha gente en los años 60.

Lo que pasó con la región pequeña. Con la bola de radio $r = 0,1$ la historia cambia por completo. RANDU estima 0,000654 cuando el valor real es 0,000524: un sesgo relativo de +24.9 %, o sea que se equivoca por una cuarta parte del valor. Su intervalo de confianza, [0,000619, 0,000689], **no contiene** el valor verdadero, y la desviación es de 7.2 errores estándar, así que esto no le pasó por mala suerte. Con exactamente la misma cantidad de ternas, el Mersenne Twister estima 0,000532, con un sesgo de +1.6 % y una desviación de 0.5 errores estándar, que es indistinguible de cero. Mismo código, mismo estimador, misma N : lo único que cambió fue quién genera los números.

El panel central de la figura 16 muestra el síntoma que delata todo: la curva de RANDU baja al principio y después **se aplana**, mientras que la del Mersenne Twister sigue la referencia $N^{-1/2}$ hasta el final. Y eso es un diagnóstico bastante claro, porque un error que deja de bajar cuando uno agrega muestras ya no es ruido, es sesgo. La diferencia importa: el ruido se arregla con más muestras y el sesgo no. RANDU no está convergiendo despacio hacia el valor correcto, está convergiendo muy bien hacia un valor equivocado.

Por qué pasa esto, geométricamente. La respuesta es la figura 11 de la Parte 1, ni más ni menos. Las ternas de RANDU no viven en el cubo, viven en 15 planos paralelos de normal $(9, -6, 1)$.

Entonces el estimador de Monte Carlo no está midiendo el volumen de la región: está midiendo qué fracción de *esos planos* cae dentro de la región, que es otra cosa.

Cuando la región es grande, como el octante de la bola, la cruzan varios planos y los errores de unos y otros se compensan al promediar, así que el resultado sale bien casi de milagro. Cuando la región es chica comparada con la separación entre planos, como la bola de radio 0,1, apenas unos pocos planos la atraviesan y la estimación queda a merced de dónde exactamente caen esos planos. Si pasan cerca del centro, sobra volumen aparente, y por eso el resultado sale +24.9 % arriba. La convergencia es real, pero converge a la medida de la intersección de los planos con la bola, que no es el número que se quería calcular.

Y los intervalos de confianza también se descomponen. La última columna de la tabla 19 repite cada estimación con 100 semillas distintas y cuenta cuántos de esos intervalos del 95 % contienen al valor real. Con el octante los dos generadores se portan bien (96 % y 97 %, que es justo lo que uno esperaría). Con la bola pequeña, el Mersenne Twister se mantiene en 93 % y RANDU se cae a 87 %. Esto es lo más grave de todo el experimento: con RANDU los intervalos dejan de significar lo que dicen significar. Están reportando una precisión que el estimador no tiene, y quien lea el resultado no tiene forma de enterarse.

Cómo se conecta todo esto con la Parte 1. Con esto se cierra el círculo que quedó abierto en la sección 2.6. Las cuatro pruebas de hipótesis miraban un valor o una pareja a la vez, y por eso le dieron el visto bueno a RANDU. La prueba espectral, que sí mira ternas, lo reprobó de inmediato. Y el cálculo de Monte Carlo de esta sección es un problema genuinamente tridimensional, así que era de esperarse que repitiera el veredicto de la prueba espectral y no el de las pruebas de hipótesis. La correspondencia no es casualidad, y de ahí sale una regla práctica bastante útil: la herramienta con la que uno diagnostica un generador tiene que mirar la misma dimensión en la que lo va a usar.

4. Conclusiones

De todo lo anterior salen cuatro conclusiones.

Uniformidad e independencia son cosas distintas, y la difícil es la segunda. Los cinco generadores se dividen en tres grupos según cómo fallan. Cuadrados medios falla de la forma más burda que existe: se muere. LCG, Mersenne Twister y Blum Blum Shub pasan las cuatro pruebas, cada uno con su propio trato entre velocidad, período y seguridad. Y RANDU también las pasa, con p -valores de lo más respetables, mientras esconde una identidad algebraica que lo tiene encerrado en 15 planos. Entonces una batería de pruebas se lee distinto de como uno esperaría: que un generador pase cuatro pruebas unidimensionales no es evidencia de que sea bueno, sino de que no se le encontró el defecto con las herramientas que se usaron, que no es lo mismo.

La teoría es la que dice dónde mirar. El defecto de RANDU no apareció explorando datos, apareció haciendo el álgebra de $a^2 \equiv 6a - 9$. Con Blum Blum Shub pasó igual: la diferencia entre un generador que funciona y uno que cicla en treinta estados no estaba en el código sino en cómo se factoriza $\lambda(\lambda(N))$, y eso no se descubre graficando. Y el período de $2^{19937} - 1$ del Mersenne Twister no es algo que alguien haya medido, es una afirmación sobre la primitividad de un polinomio. En los tres casos, primero está la matemática y después el experimento.

Monte Carlo es lento, pero le da igual la dimensión. La tasa $\mathcal{O}(N^{-1/2})$ que derivamos y verificamos es mala comparada con cualquier cuadratura en una dimensión, y aun así no depende de d . Cuando uno ve que la rejilla necesita 10^{20} puntos para tener diez subdivisiones por eje en veinte dimensiones, esa indiferencia deja de ser un detalle técnico y se vuelve la única razón por la que el cálculo se puede hacer. Eso sí, la dimensión igual cobra lo suyo a través de σ , y ahí entran las técnicas de reducción de varianza: bajar la constante salió hasta 318.55 veces más rentable que subir N , pero sólo cuando la técnica le queda al integrando. Las antitéticas aplicadas a una función simétrica no ayudaron nada, al contrario, costaron el doble.

Un método de Monte Carlo nunca va a ser mejor que su fuente de aleatoriedad. El experimento reproduce esta moraleja de forma bastante limpia. Con una región grande, RANDU da un resultado correcto y tranquilizador. Con una región chica, el mismo generador, el mismo estimador y el mismo número de muestras dan un error de +24.9% que no baja aunque uno agregue muestras, y un intervalo de confianza que miente. Y lo más preocupante no es que RANDU falle, porque eso se sabe desde 1968, sino que falle *en silencio*: sin la fórmula cerrada para comparar, nada en la salida del programa habría dicho que ese número estaba mal.

Referencias

Referencias

- [1] G. Marsaglia, “Random Numbers Fall Mainly in the Planes”, *Proceedings of the National Academy of Sciences*, vol. 61, no. 1, pp. 25–28, 1968. doi: 10.1073/pnas.61.1.25.
- [2] RANDOM.ORG, *Introduction to Randomness and Random Numbers*. <https://www.random.org/randomness/>
- [3] M. Matsumoto y T. Nishimura, “Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator”, *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, 1998.
- [4] L. Blum, M. Blum y M. Shub, “A Simple Unpredictable Pseudo-Random Number Generator”, *SIAM Journal on Computing*, vol. 15, no. 2, pp. 364–383, 1986.
- [5] T. E. Hull y A. R. Dobell, “Random Number Generators”, *SIAM Review*, vol. 4, no. 3, pp. 230–254, 1962.
- [6] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3a ed., Addison-Wesley, 1997.
- [7] W. H. Press, S. A. Teukolsky, W. T. Vetterling y B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3a ed., Cambridge University Press, 2007.
- [8] A. B. Owen, *Monte Carlo Theory, Methods and Examples*, 2013.